History of Java

•JAVA was originated at Sun Microsystems Inc.

• It was conceived by James Gosling, Patrick Naughton, Chris Warth, Ed Frank, and Mike Sheridan in 1991

• The goal was to develop a device independent language for various electronic hardware. Once they wrote the language they decided to go for a full developed language.

• The goal was to remove the obstacles of existing languages like C++ and C History of Java

• It took 18 months to develop the first working version. This language was initially called “Oak,” but was renamed “Java” in 1995

• It was officially released in 1996 as Java 1.0

• Subsequently improved as Java 2.0

• Simple programming language to write, compile and debug a program easily.

• Emergence of World wide web (www) helped Java to flourish (better)

JVM:

Java Virtual Machine

Definition:
The JVM is the heart of Java's "Write once, run anywhere" capability.

It's an abstract computing machine that executes Java bytecode.

Functions:

- **Loads Code:** It loads bytecode into memory.
- **Verifies Code:** It checks the bytecode for safety and security.
- **Executes Code:** It interprets or compiles bytecode into machine code that the underlying operating system can understand.

- **Provide Runtime Environment:** It manages memory, threads, and garbage collection during program execution.
- **Platform dependent:** Each operating system has its own specific JVM implementation.
    - This is why you need a different JVM installation for each OS.

Analogy:
Think of the JVM as a universal translator that understands Java bytecode and converts it into the language of the specific computer it is running on.

JRE:

Java Runtime Environment

Definition:
The JRE provides everything needed to run Java programs.

- It's a software package that includes the JVM along with libraries and other components required to execute Java applications.

~ Contents:

- **JVM:** As explained above.
- **Java Class Libraries:** Pre-written code that provides common functionalities.
- **Other Supporting Files:** Files needed by the JVM at runtime.

~ Use Case:
If you only want to run Java programs, you only need the JRE.

~ Analogy:
JRE is a complete toolkit. It includes JVM, libraries, and other tools.

JDK:

Java Development Kit

Definition:

- The JDK is a complete development environment for building Java applications.

- It includes the JRE as well as compilers, debuggers, and development tools.

~ Contents:

- **JRE:** Includes everything in the JRE.
- **Development Tools:**
    - Java C: It is a compiler, which translates Java source files into a class file (bytecode).
    - jdb: The Java debugger, for finding and fixing errors in code.
    - Javadoc: A tool for generating API documentation from Java source code.
    - Other useful files.

~ Use Case:
If you want to develop Java applications, you need the JDK.

~ Analogy:
JDK has all the tools in the JRE plus extra tools for developers. It's like having a workshop with all the tools to build and test a product.

~ Note:

- **JVM:** Executes bytecode.
- **JRE:** Runs Java programs.
- **JDK:** Develops and runs Java programs.

~ Summary:

- The JDK contains the JRE, and the JRE contains the JVM.

~ Remember Conclusion:

- **JDK** *(Development)*
- **JRE** *(Execution)*
- **JVM** *(Execution*

Java Platforms

Java provides four different platforms, each designed for specific application development. These platforms offer a range of tools, APIs, and environments to suit different types of applications.

✅ **1. Java Standard Edition (Java SE)**

- **Purpose:** Develops and runs general-purpose desktop and server-side applications.
- **Features:**
    - Core Java libraries (e.g., java.lang, java.util, java.io)
    - Networking, Security, and Database access APIs
    - Multithreading and Collection Framework
- **Use Case:** Ideal for standalone applications like desktop software or backend systems.

✅ **2. Java Enterprise Edition (Java EE)** ***(Now Jakarta EE)***

- **Purpose:** Builds large-scale, distributed, and multi-tier enterprise applications.
- **Features:**
    - Servlets, JSP (JavaServer Pages), and WebSocket APIs
    - EJB (Enterprise Java Beans) for business logic implementation
    - Support for RESTful and SOAP web services
- **Use Case:** Suitable for complex web applications, financial systems, and e-commerce platforms.

✅ **3. Java Micro Edition (Java ME)**

- **Purpose:** Develops mobile, embedded, and IoT (Internet of Things) applications.
- **Features:**
    - Optimized for limited-resource devices like mobile phones and sensors
    - Lightweight libraries and runtime environment
- **Use Case:** Commonly used in feature phones, set-top boxes, and smart appliances.

Java Class File

A **Java Class File** is a file with the extension .class that contains the bytecode generated by the Java Compiler (javac). It is a platform-independent file that the **Java Virtual Machine (JVM)** can interpret and execute.

✅ How is a Class File Created?

1. **Write Java Code:**
    - First, you write a Java source code file with the .java extension.
2. **Compile the Code:**
    - The javac compiler compiles the .java file into a .class file.
    - Example:

bash Copy Edit

```
javac MyProgram.java
```

This will generate `MyProgram.class`.

3. **Execute the Class File:**
    - The JVM runs the .class file using the java command:

bash Copy Edit

```
java MyProgram
```

✅ Contents of a Class File

A .class file contains:

- **Bytecode:** Machine-independent instructions that the JVM can interpret.
- **Metadata:** Information about the class, methods, fields, and constants.
- **Version Information:** The version of Java used for compilation.
- **Constant Pool:** Stores literals, class references, and method references.
- **Methods and Fields:** Includes method definitions and variable details.

C++ and Java comparison

Feature	C++	Java
Paradigm	C++ is a procedural and object-oriented programming language.	Java is primarily an object-oriented programming language.
Platform	C++ is platform-dependent, meaning it needs to be recompiled for each platform.	Java is platform-independent due to its "Write Once, Run Anywhere" capability.
Memory Management	C++ requires manual memory management using pointers and memory allocation.	Java provides automatic memory management using garbage collection.
Pointers	C++ supports pointers and allows direct memory access.	Java does not support explicit pointers, providing better security and memory management.
Multiple Inheritance	C++ supports multiple inheritance, allowing a class to inherit from more than one class.	Java supports single inheritance but allows multiple inheritance using interfaces.
Operator Overloading	C++ supports operator overloading, which allows operators to be redefined for user-defined data types.	Java does not support operator overloading to simplify code and reduce complexity.
Goto Statement	C++ supports the `goto` statement for unconditional jumps in code.	Java does not support the `goto` statement as it can lead to unreadable and unmaintainable code.
Performance	C++ generally provides faster performance because it is closer to machine code.	Java is generally slower due to the additional layer of the JVM, but Just-In-Time (JIT) compilation has improved its performance significantly.

Features of Java.

Java is a powerful and versatile programming language with a rich set of features that have contributed to its widespread adoption.

Here are some of its key features:

1. Object-Oriented

Java is a true object-oriented programming (OOP) language. This means that everything in Java is an object, and programs are designed using classes and objects based on real-world concepts.

Key OOP Features supported by Java:

- Encapsulation

- Inheritance
- Polymorphism
- Abstraction

2. Platform Independent

Java follows the "write once, run anywhere" (WORA) capability, which is one of its most significant features. Java code is compiled into bytecode, which is platform-independent and intermediate language.

3. Simple and Easy to Learn

Java has an absolutely simple syntax compared to languages like C++. It avoids complex features like pointers and manual memory management, making it easier to learn and use.

4. Robust

Java is designed to be robust, meaning it can handle errors and unexpected situations gracefully. It has strong type-checking, exception handling mechanisms, and automatic garbage collection, which helps prevent memory leaks and common programming errors.

5. Secure

Security is a primary concern in Java's design. Bytecode verification keeps malicious code from executing.

6. Portable

Java bytecode is portable across different operating systems and hardware architectures, allowing applications to run without modification.

7. Dynamic

Java supports dynamic memory allocation and loading of classes at runtime, making it highly adaptive and flexible.

8. Multithreaded

Java has built-in support for multithreading, enabling the simultaneous execution of two or more threads for efficient performance.

9. Distributed

Java is designed for distributed computing, supporting the development of network-centric applications.

10. Extensible

Java's support for libraries and frameworks makes it highly extensible. There is a vast ecosystem of open-source and commercial libraries available for various tasks, such as web development, data processing, and machine learning.

Installing java development kit step simple

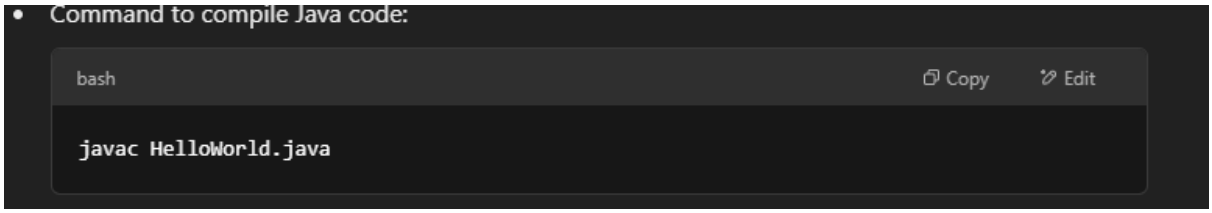
Here are the simple steps to install the Java Development Kit (JDK):

1. **Download the JDK**:
    - Visit the official Oracle website or the OpenJDK website.
    - Choose the appropriate version for your operating system (Windows, macOS, or Linux).
    - Download the installer.
2. **Install the JDK**:
    - Run the downloaded installer.
    - Follow the on-screen instructions to complete the installation.
    - Note the installation directory (default is usually C:\Program Files\Java\jdk-xx on Windows).
3. **Set Environment Variables (for Windows)**:
    - Go to **System Properties** → **Environment Variables**.
    - Under **System Variables**, select Path and click **Edit**.
    - Add the bin directory of the JDK (e.g., C:\Program Files\Java\jdk-xx\bin).
    - Create a JAVA_HOME variable and set it to the JDK directory.

Java Compiler and **Java Interpreter**

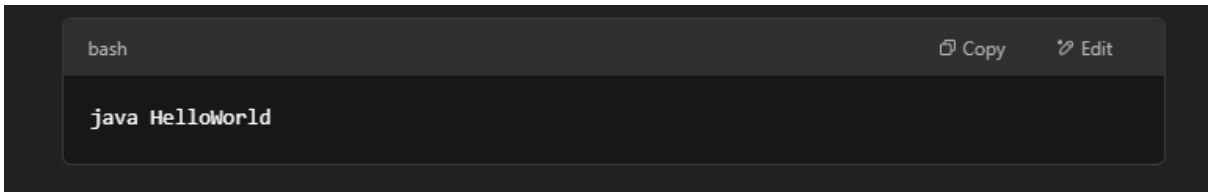
Java Compiler (javac)

- The Java Compiler is a tool that converts **Java source code** (.java files) into **bytecode** (.class files).
- It checks for syntax errors and ensures the code follows Java language rules.
- The generated bytecode is platform-independent and can run on any system with a Java Virtual Machine (JVM).
- Command to compile Java code:



Java Interpreter (java)

- The Java Interpreter is part of the JVM that reads and executes the compiled bytecode line by line.
- It converts the bytecode into machine code specific to the host operating system.
- Unlike compiled languages that generate machine-specific code, Java remains platform-independent through the interpreter.
- Command to run Java code:



In Short

- **Java Compiler** → Converts source code to bytecode.
- **Java Interpreter** → Executes the bytecode on JVM.

Using CLASSPATH in Java

CLASSPATH is an environment variable in Java that tells the Java compiler (javac) and Java interpreter (java) where to find .class files, JAR files, or other resources needed for your program.

Purpose of CLASSPATH

- It helps the JVM locate user-defined classes and external libraries.
- It ensures that Java programs run smoothly without class not found errors.

 Windows:

- Go to **System Properties** → **Environment Variables**.
- Find or create CLASSPATH.
- Add the paths to .class files or JAR files.

Main function in java

Explain the main Function of Java
The main Function in Java is the entry point of any standalone Java application.

- It's where the Program execution begins.

Here's a breakdown of its key aspects:

Signature:
The main function in Java must have a specific signature:



1. **Public:**
    - The public keyword makes the main method accessible from outside the class.
    - It needs to be public because the Java Virtual Machine (JVM) needs to be able to access it to start the program.

2. **Static:**
    - The static keyword means that the main method belongs to the class itself, rather than to a specific instance of the class.
    - This is important because the JVM needs to be able to call the main method without creating an object of the class.
3. **Void:**
    - The void keyword indicates that the main method does not return any value.
4. **main(String[] args):**
    - The main method takes an array of Strings as its argument.
    - This array, args, is used to pass command-line arguments to the program.
    - Each element in the args array represents a separate command-line argument.

How It Works?

- When you run a Java program, the JVM looks for the main method with the exact signature public static void main(String[] args).
- The JVM then starts executing the code within the main method.
- The main method can call other methods, create objects, and perform any other actions necessary for the program to run.
- Once the main method finishes executing, the program terminates.

Example:

java Copy Edit

```java
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello World!!");
    }
}
```

In this example, the main method prints "Hello World!!" to the console.

Why is main function declared as static?

The main Function in Java is declared as static for a very specific reason — to allow the Java Virtual Machine (JVM) to call it directly without needing to create an instance of the class.

Let's break down why this is essential:

1. **Entry Point for Execution:**
    - The main method serves as the entry point for any Java application.
    - When you run a Java program, the JVM needs a place to start execution.
    - This starting point is the main method.
2. **No Object Exists Initially:**
    - When the JVM begins running a Java program, no objects of any classes have been created yet.
    - Objects are created through constructors, which are instance methods.
3. **Calling Without an Object:**
    - If the main method were not static, it would be an instance method.
    - This would mean that the JVM would need to create an object of the class containing the main method before it could call the method.
    - However, at the very beginning of program execution, there are no objects.
4. **The Role of Static:**
    - The static keyword makes a method belong to the class itself, rather than to any specific instance of the class.
    - This means that you can call a static method directly using the class name, without needing to create an object.

Example:

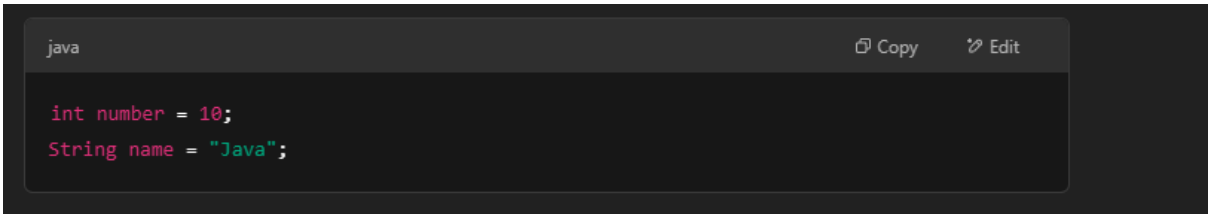
java Copy Edit

```java
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

- In this example, the main method is static.
- The JVM can call it directly using Main.main(args) without needing to create an object of the Main class.

Java Statements

1. **Declaration Statements**
    - Used to declare variables and constants.



2. **Expression Statements**
    - Perform operations like assignments, method calls, and object creations.



3. **Control Flow Statements**
    - Manage the flow of a program using:
    - **Conditional Statements** (if, if-else, switch)
    - **Looping Statements** (for, while, do-while)
    - **Jump Statements** (break, continue, return)

4. **Synchronized Statements**

- Ensure thread safety by synchronizing blocks of code.

```java
synchronized (this) {
    System.out.println("Thread-safe block");
}
```

5. **Try-Catch Statements**
    - Handle exceptions and prevent program crashes.

```java
try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Error: " + e.getMessage());
}
```

6. **Labeled Statements**
    - Provide labels to loops for better control using break or continue.

```java
outerLoop:
for (int i = 0; i < 5; i++) {
    for (int j = 0; j < 5; j++) {
        if (j == 3) {
            break outerLoop;
        }
        System.out.println(i + ", " + j);
    }
}
```

Data Types in Java

In Java, data types specify the size and type of values that variables can store. They are divided into two main categories:

1. Primitive Data Types

- **byte**: 8-bit integer, used to save memory.
- **short**: 16-bit integer.
- **int**: 32-bit integer, commonly used for numeric values.
- **long**: 64-bit integer, used when a larger range is needed.
- **float**: 32-bit floating-point number, for decimal values.
- **double**: 64-bit floating-point number, for precise decimal values.
- **char**: 16-bit Unicode character.
- **boolean**: Represents true or false.

2. Non-Primitive Data Types

- **String**: A sequence of characters.
- **Arrays**: Collection of similar data types.
- **Classes**: Custom data types defined by the user.
- **Interfaces**: Define abstract data types.

Each data type serves a specific purpose in Java, allowing for efficient memory management and accurate data representation.

Variables in Java with Examples

A **variable** in Java is a container that holds data during program execution. It is associated with a data type, which determines what kind of data it can store.

Types of Variables in Java

1. **Local Variables**
    - Declared inside a method, constructor, or block.
    - Accessible only within the scope in which they are defined.

- Example:

```java
public class Example {
    public void display() {
        int localVar = 10; // Local variable
        System.out.println("Local Variable: " + localVar);
    }
}
```

2. **Instance Variables**
    - Declared inside a class but outside any method.
    - Belong to an object and are created when the object is instantiated.
    - Example:

```java
public class Example {
    int instanceVar = 20; // Instance variable

    public void display() {
        System.out.println("Instance Variable: " + instanceVar);
    }
}
```

3. **Static Variables**
    - Declared using the static keyword inside a class.
    - Belong to the class rather than any specific object.
    - Shared by all instances of the class.

- Example:

```java
public class Example {
    static int staticVar = 30; // Static variable

    public static void display() {
        System.out.println("Static Variable: " + staticVar);
    }
}
```

These examples demonstrate how different types of variables work in Java.

Literals in Java

- **Character Literals:**

  Character values are specified using standard C notation, with extensions for Unicode values:

```java
char c = 'a';       // Character lowercase 'a'
char d = '\n';      // Newline character
char e = '\u2122';  // Unicode character (™)
```

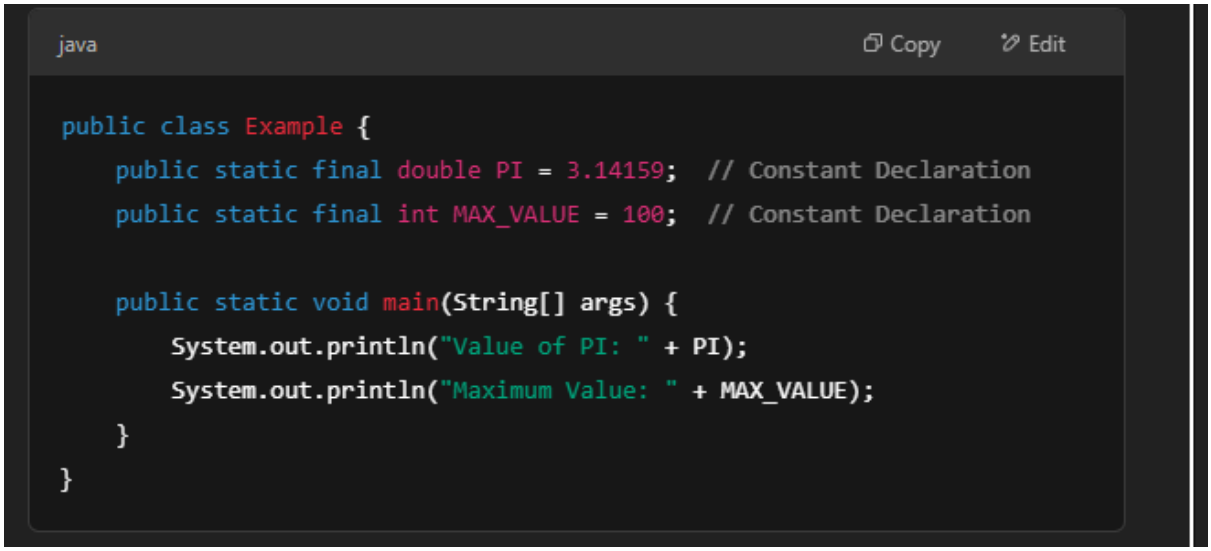
- **Boolean Literals:**

  The boolean values represent `true` or `false`:

```java
boolean ready = true;  // Boolean value true
boolean late = false;  // Boolean value false
```

Constants in Java

- A **constant** is a variable whose value cannot be changed once assigned.
- Constants are declared using the final keyword in Java.
- They are usually written in uppercase letters with underscores separating words.



 final ensures that the value cannot be modified once it is assigned.

 Constants provide clarity and prevent accidental changes in values.

Expressions in Java

An **expression** in Java is a combination of variables, constants, operators, and method calls that produce a single value. It is a fundamental unit of code that performs a calculation or an operation.

Types of Expressions in Java

1. **Arithmetic Expressions**
    - Perform mathematical operations like addition, subtraction, multiplication, division, and modulus.
    - Example: int result = a + b * c;
2. **Relational Expressions**
    - Compare values using relational operators (>, <, >=, <=, ==, !=).

- Example: boolean isGreater = a > b;

3. **Logical Expressions**
    - Combine multiple conditions using logical operators (&&, ||, !).
    - Example: boolean result = (a > b) && (b < c);
4. **Bitwise Expressions**
    - Perform operations at the bit level using bitwise operators (&, |, ^, ~).
    - Example: int result = a & b;
5. **Assignment Expressions**
    - Assign values to variables using assignment operators (=, +=, -=, *=, /=).
    - Example: x += 10;
6. **Conditional (Ternary) Expressions**
    - Use the ternary operator ? : for concise conditional evaluation.
    - Example: int max = (a > b) ? a : b;
7. **Instanceof Expressions**
    - Check whether an object is an instance of a particular class or subclass.
    - Example: boolean isString = obj instanceof String;

Expressions are evaluated following Java's operator precedence and produce a result that can be stored in a variable or used directly.

Arrays in java:

1D and 2D Arrays in Java

1D Array (One-Dimensional Array)

- A 1D array is a collection of elements arranged in a single row.
- It is declared and initialized using square brackets `[]`.
- All elements in the array must be of the same data type.

Declaration and Initialization:

java Copy Edit

```java
int[] numbers = new int[5]; // Declaration of a 1D array with 5 elements
int[] values = {10, 20, 30, 40, 50}; // Initialization with values
```

Accessing and Modifying Elements:

java Copy Edit

```java
System.out.println(values[2]); // Accesses the 3rd element (30)
values[1] = 25; // Modifies the 2nd element (20 to 25)
```

2D Array (Two-Dimensional Array)

- A 2D array is an array of arrays, arranged in rows and columns like a matrix.
- It uses two sets of square brackets `[][]`.
- All elements in the array must be of the same data type.

Declaration and Initialization:

java Copy Edit

```java
int[][] matrix = new int[3][3]; // Declaration of a 3x3 matrix
int[][] numbers = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
}; // Initialization with values
```

Accessing and Modifying Elements:

java Copy Edit

```java
System.out.println(numbers[1][2]); // Accesses element at 2nd row, 3rd columr
numbers[0][1] = 10; // Modifies element at 1st row, 2nd column (value 2 to 10
```

- 1D Arrays are useful for storing linear data, such as a list of student grades.
- 2D Arrays are useful for representing tabular data like matrices, tables, or game boards.

Primitive Wrapper Classes in Java

In Java, **Wrapper Classes** provide an object representation of the eight primitive data types. They are part of the java.lang package and offer useful methods to manipulate and convert data.

Each primitive data type has a corresponding wrapper class:

Primitive Type	Wrapper Class
`byte`	`Byte`
`short`	`Short`
`int`	`Integer`
`long`	`Long`
`float`	`Float`
`double`	`Double`
`char`	`Character`
`boolean`	`Boolean`

Why Use Wrapper Classes?

1. **Object Representation:**
    - Primitive data types cannot be used in collections like ArrayList, HashMap, etc.
    - Wrapper classes allow primitives to be used as objects.
2. **Utility Methods:**
    - Provide useful methods for converting between data types and performing operations.
3. **Null Support:**
    - Unlike primitives, wrapper objects can be assigned null values.
4. **Autoboxing and Unboxing:**
    - Java automatically converts between primitives and their corresponding wrapper objects.

Example of Wrapper Classes

java Copy Edit

```java
public class WrapperExample {

    public static void main(String[] args) {
        // Primitive to Wrapper (Autoboxing)
        int num = 10;
        Integer wrappedNum = num;
        System.out.println("Wrapped Integer: " + wrappedNum);

        // Wrapper to Primitive (Unboxing)
        Double wrappedDouble = 20.5;
        double primitiveDouble = wrappedDouble;
        System.out.println("Primitive Double: " + primitiveDouble);

        // Using Methods of Wrapper Classes
        String str = "123";
        int parsedInt = Integer.parseInt(str);
        System.out.println("Parsed Integer: " + parsedInt);
    }
}
```

Class and Object in Java

Class in Java

- A **class** in Java is a blueprint or template for creating objects.
- It defines the **properties (attributes)** and **behavior (methods)** of an object.
- A class contains **fields** (variables), **methods**, **constructors**, and sometimes **inner classes**.

Example:

java Copy Edit

```java
// Class Declaration
public class Car {

    // Fields (Properties)
    String brand;
    String model;
    int year;

    // Method (Behavior)
    void displayDetails() {
        System.out.println("Brand: " + brand);
        System.out.println("Model: " + model);
        System.out.println("Year: " + year);
    }
}
```

Object in Java

- An **object** is an instance of a class.
- It represents a real-world entity with a state (values of attributes) and behavior (methods).
- Using the new keyword, objects are created from a class.

java Copy Edit

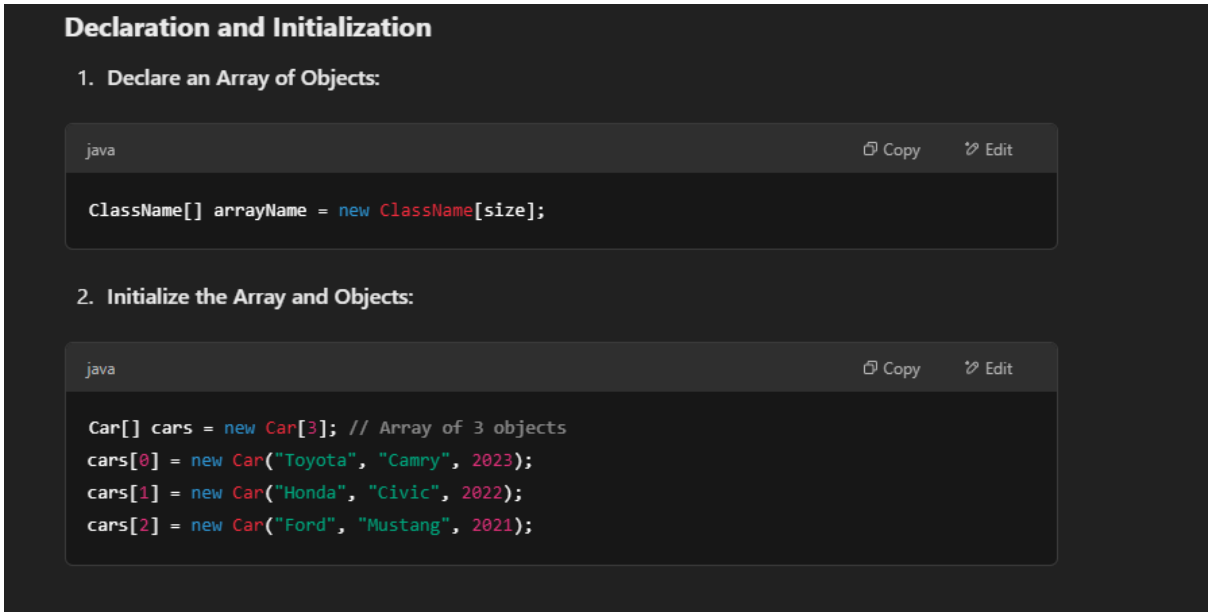
```java
public class Main {
    public static void main(String[] args) {
        // Creating an object of the Car class
        Car car1 = new Car();
        car1.brand = "Toyota";
        car1.model = "Camry";
        car1.year = 2023;

        // Calling method using the object
        car1.displayDetails();
    }
}
```

Array of objects

Array of Objects in Java

- An **array of objects** in Java is a collection of object references stored in a fixed-size, indexed structure.
- It allows you to store multiple objects of the same class in a single variable.
- Each element of the array points to an object of the specified class.





AutoBoxing and Unboxing in Java

Autoboxing and **Unboxing** are the processes of converting between **primitive data types** and their corresponding **wrapper classes** in Java.

- **Autoboxing:** Automatic conversion of a **primitive data type** to its corresponding **wrapper class**.
- **Unboxing:** Automatic conversion of a **wrapper class** object to its corresponding **primitive data type**.

These conversions are handled automatically by the Java Compiler, reducing the need for manual conversion.

✅ Autoboxing

- Converts **primitive types** to their respective **wrapper classes**.
- Occurs when a primitive is assigned to a variable of a wrapper class or when passed as an object parameter.

```java
int num = 10;
Integer wrappedNum = num;  // Autoboxing: int → Integer
System.out.println("Autoboxed Integer: " + wrappedNum);
```

✅ Unboxing

- Converts **wrapper class objects** back to their corresponding **primitive types**.
- Occurs when a wrapper class object is assigned to a primitive variable.

```java
Integer objNum = 50;
int primitiveNum = objNum;  // Unboxing: Integer → int
System.out.println("Unboxed Primitive: " + primitiveNum);
```

✅ Example of Autoboxing and Unboxing Together

```java
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> numbers = new ArrayList<>();

        // Autoboxing: Converting int to Integer (Wrapper Class)
        numbers.add(100);
        numbers.add(200);

        // Unboxing: Converting Integer to int
        int result = numbers.get(0) + numbers.get(1);
        System.out.println("Sum: " + result);
    }
}
```

ArrayList in Java

- **ArrayList** is a part of the java.util package and provides a dynamic array that can grow or shrink in size.
- Unlike arrays, ArrayList allows flexible resizing and offers easy methods for manipulation.
- It can store objects, including custom objects.

Accessing with For-Each Loop

Example:

java Copy Edit

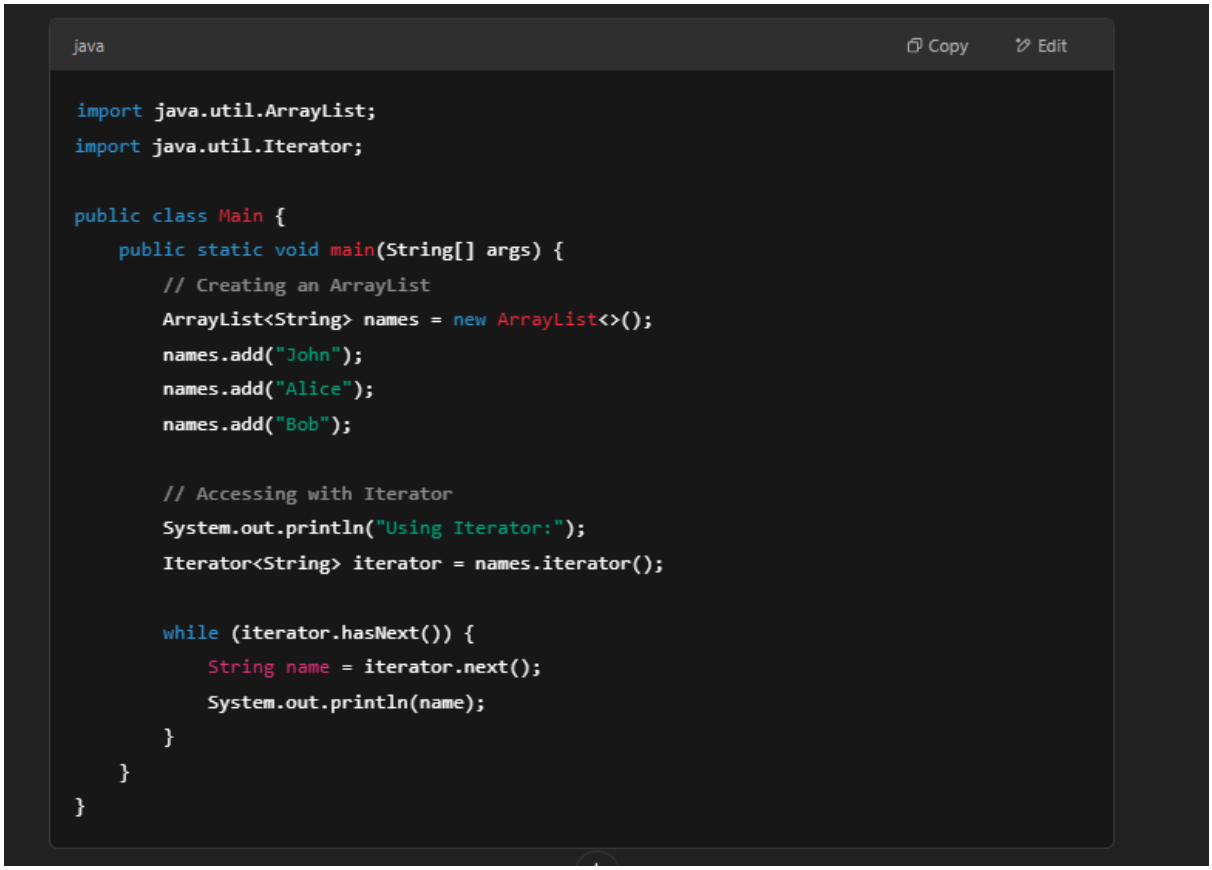
```java
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        // Creating an ArrayList
        ArrayList<String> names = new ArrayList<>();
        names.add("John");
        names.add("Alice");
        names.add("Bob");

        // Accessing with For-Each Loop
        System.out.println("Using For-Each Loop:");
        for (String name : names) {
            System.out.println(name);
        }
    }
}
```

Accessing with Iterator

- **Iterator** is an interface used to iterate over collections like ArrayList.
- It provides methods like hasNext() and next() for traversing the list.
- You can also remove elements safely using Iterator.remove().



Vectors in Java

- **Vector** is a class in the java.util package that implements the **List interface**.
- It is similar to an **ArrayList** but is **synchronized**, meaning it is thread-safe (multiple threads can access it without issues).
- It dynamically grows as elements are added.

✅ **Features of Vector**

- **Synchronized**: Thread-safe for multi-threaded applications.
- **Resizable**: Can grow or shrink dynamically as needed.
- **Random Access**: Allows fast access using indexes.
- **Contains Duplicates**: Like ArrayList, it permits duplicate elements.

✅ Creating a Vector in Java

You can create a vector using the following constructors:

1. **Default Constructor:**

java Copy Edit

```java
Vector<String> vector = new Vector<>();
```

2. **With Initial Capacity:**

java Copy Edit

```java
Vector<Integer> vector = new Vector<>(20); // Initial capacity is 20
```

3. **With Capacity Increment:**

java Copy Edit

```java
Vector<Double> vector = new Vector<>(10, 5); // Initial capacity 10, increments by 5
```

```java
// put your co// Class Definition
import java.util.Vector;

public class Main {
    public static void main(String[] args) {
        // Creating a Vector
        Vector<String> vector = new Vector<>();

        // Adding Elements
        vector.add("Apple");
        vector.add("Banana");
        vector.add("Orange");

        // Accessing Elements
        System.out.println("First Element: " + vector.get(0));

        // Iterating using For-Each Loop
        System.out.println("Vector Elements:");
        for (String fruit : vector) {
            System.out.println(fruit);
        }

        // Removing an Element
        vector.remove("Banana");
        System.out.println("After Removing Banana: " + vector);

        // Checking Size and Capacity
        System.out.println("Vector Size: " + vector.size());
        System.out.println("Vector Capacity: " + vector.capacity());
    }
}
```

snappify.com

String and String Buffer class

✅ 1. String in Java

- **Immutable**: Strings in Java are **immutable**, meaning once a String object is created, it cannot be changed.
- Every time you modify a string (e.g., concatenation), a new String object is created.
- **Stored in String Pool**: Strings are stored in a special memory area called the **String pool** to optimize memory usage.

Example:

java Copy Edit

```java
String s1 = "Hello";
String s2 = s1.concat(" World"); // Creates a new String
System.out.println(s1);  // Output: Hello
System.out.println(s2);  // Output: Hello World
```

Method	Description	Example
`toUpperCase()`	Converts all characters to uppercase.	`"java".toUpperCase()` → `"JAVA"`
`toLowerCase()`	Converts all characters to lowercase.	`"JAVA".toLowerCase()` → `"java"`
`trim()`	Removes extra spaces from both ends of a string.	`" hello ".trim()` → `"hello"`
`charAt(int index)`	Returns the character at the specified index.	`"Hello".charAt(1)` → `'e'`
`substring(int beginIndex)`	Extracts a substring starting from the given index.	`"Programming".substring(3)` → `"gramming"`
`replace(char oldChar, char newChar)`	Replaces all instances of a character with another.	`"banana".replace('a', 'o')` → `"bonono"`
`length()`	Returns the number of characters in a string.	`"Java".length()` → `4`
`isEmpty()`	Checks if the string has no characters.	`"".isEmpty()` → `true`

✅ 2. StringBuffer in Java

- **Mutable**: StringBuffer objects are **mutable**, meaning they can be changed without creating a new object.
- It is more memory-efficient when performing multiple string operations.
- **Thread Safe**: StringBuffer is **synchronized** and suitable for multi-threaded environments.

Example:

java Copy Edit

```java
StringBuffer sb = new StringBuffer("Hello");
sb.append(" World"); // Appends to the same object
System.out.println(sb);  // Output: Hello World
```

Method	Description	Example
`append(String str)`	Appends the specified string to the sequence.	`new StringBuffer("Hello").append(" World")` → `"Hello World"`
`capacity()`	Returns the current capacity of the StringBuffer.	`new StringBuffer().capacity()` → `16`
`charAt(int index)`	Returns the character at the specified index.	`new StringBuffer("Java").charAt(1)` → `'a'`
`delete(int start, int end)`	Removes characters from the specified start to end index.	`new StringBuffer("Hello World").delete(5, 11)` → `"Hello"`
`deleteCharAt(int index)`	Removes the character at the specified index.	`new StringBuffer("Java").deleteCharAt(1)` → `"Jva"`
`reverse()`	Reverses the sequence of characters.	`new StringBuffer("Java").reverse()` → `"avaJ"`
`trimToSize()`	Reduces the storage used for the character sequence.	`new StringBuffer("Hello").trimToSize()` → *(Trims unused space)*

✅ Key Differences Between String and StringBuffer

Aspect	String	StringBuffer
Mutability	Immutable	Mutable
Memory Efficiency	Less efficient due to object creation	More efficient for modifications
Thread Safety	Not Thread Safe	Thread Safe (Synchronized)
Performance	Slower for large data operations	Faster due to in-place changes
Usage	Suitable for constant data	Suitable for frequently modified data

✅ **When to Use**

- **Use String** when the text remains constant or changes rarely.
- **Use StringBuffer** when you need to perform multiple string manipulations in a multi-threaded environment.
- If thread safety is not required, consider using StringBuilder, which is faster and non-synchronized.

Date and Math in Java

Math Class in Java

- The Math class in Java provides various methods for performing mathematical operations like finding maximum, minimum, square root, and trigonometric functions.

Method	Description	Example
`Math.abs(x)`	Returns the absolute value of `x`.	`Math.abs(-5)` → `5`
`Math.max(x, y)`	Returns the larger of `x` and `y`.	`Math.max(10, 20)` → `20`
`Math.min(x, y)`	Returns the smaller of `x` and `y`.	`Math.min(10, 20)` → `10`
`Math.sqrt(x)`	Returns the square root of `x`.	`Math.sqrt(16)` → `4.0`
`Math.pow(x, y)`	Returns `x` raised to the power of `y`.	`Math.pow(2, 3)` → `8.0`
`Math.random()`	Returns a random number between 0.0 and 1.0.	`Math.random()` → `0.72`
`Math.round(x)`	Rounds `x` to the nearest integer.	`Math.round(4.6)` → `5`
`Math.floor(x)`	Rounds `x` down to the nearest integer.	`Math.floor(4.9)` → `4.0`
`Math.ceil(x)`	Rounds `x` up to the nearest integer.	`Math.ceil(4.1)` → `5.0`

Date in Java

- Java provides multiple classes to handle and manipulate date and time using the java.util.Date and java.util.Calendar classes.

Common Date Classes:

Class	Description
`Date`	Represents a specific moment in time.
`Calendar`	Provides methods to manipulate date and time.
`LocalDate`	Represents a date without a time.
`LocalTime`	Represents a time without a date.
`LocalDateTime`	Represents both date and time.

Example:

java Copy Edit

```java
import java.util.Date;

public class DateExample {
    public static void main(String[] args) {
        Date date = new Date();
        System.out.println("Current Date: " + date);
    }
}
```

1. Equality (==)

- **Definition**: Equality refers to checking whether two references point to the **same object** in memory.
- It is a comparison of memory addresses (object references).
- Used with **primitive data types** like int, char, double, etc.
- For objects, == checks if two object references refer to the **same instance**.

Example (Equality with Primitives)

java Copy Edit

```java
int a = 10;
int b = 10;
System.out.println(a == b); // true (Values are the same)
```

Example (Equality with Objects)

java Copy Edit

```java
String str1 = new String("Java");
String str2 = new String("Java");
System.out.println(str1 == str2); // false (Different objects)
```

2. Equivalence (equals() Method)

- **Definition**: Equivalence refers to checking whether two objects have the **same value** or content.
- The equals() method is defined in the Object class and is often overridden to provide meaningful value comparison.
- Especially used for non-primitive data types like String, Integer, List, etc.

Example (Equivalence with Strings)

java Copy Edit

```java
String str1 = new String("Java");
String str2 = new String("Java");
System.out.println(str1.equals(str2)); // true (Content is the same)
```

Unit 2:

Inheritance in Java

Inheritance is one of the core concepts of **Object-Oriented Programming (OOP)** in Java. It allows a class (child class) to inherit properties and behaviors (methods) from another class (parent class).

✅ **Key Features of Inheritance:**

- Promotes **code reusability**.
- Establishes a **parent-child relationship**.
- Supports the **hierarchical structure** of classes.
- Allows for **method overriding** and **polymorphism**.

Types of Inheritance in Java

Java supports the following types of inheritance:

1. **Single Inheritance**:
    - One class inherits from one parent class.
    - Example: Class B extends Class A
2. **Multilevel Inheritance**:
    - A class inherits from a class which itself is inherited from another class.
    - Example: Class C extends Class B → Class B extends Class A
3. **Hierarchical Inheritance**:
    - Multiple child classes inherit from a single parent class.
    - Example: Class B extends Class A, Class C extends Class A
4. **Hybrid Inheritance** *(Not directly supported)*:
    - Combination of multiple types of inheritance using **interfaces**.

Note: Java does **not support multiple inheritance** using classes to avoid ambiguity. However, it can be achieved using **interfaces**.

- To create inheritance, use the extends keyword



Polymorphism in Java

Polymorphism is one of the core concepts of Object-Oriented Programming (OOP) in Java. The word "polymorphism" means **"many forms"**. In Java, polymorphism allows objects to behave in different ways depending on their class type.

✅ Types of Polymorphism in Java

There are **two types of polymorphism** in Java:

1. **Compile-Time Polymorphism (Method Overloading)**
2. **Runtime Polymorphism (Method Overriding)**

📌 1. Compile-Time Polymorphism (Method Overloading)

- Achieved using **Method Overloading**.
- Multiple methods can have the **same name** but with different parameters (type, number, or both).
- Method resolution is done at **compile tim**

Example of Method Overloading:

java Copy Edit

```java
class MathOperations {
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}

public class Main {
    public static void main(String[] args) {
        MathOperations obj = new MathOperations();
        System.out.println(obj.add(5, 10));          // Calls int version
        System.out.println(obj.add(5.5, 10.5));      // Calls double version
        System.out.println(obj.add(5, 10, 15));      // Calls 3 int version
    }
}
```

📌 2. Runtime Polymorphism (Method Overriding)

- Achieved using **Method Overriding**.
- A **subclass** provides its own implementation of a method that is already defined in its **parent class**.
- Method resolution happens at **runtime** using **dynamic method dispatch**.
- **@Override** annotation is used to ensure proper overriding.



Abstract Classes in Java

An **abstract class** in Java is a class that **cannot be instantiated** (meaning you cannot create objects of it). It is primarily used as a **base class** for other classes to inherit common properties and behaviors.

Abstract classes are often used to define a **template** or **blueprint** for other classes.

✅ Features of Abstract Classes

- Declared using the abstract keyword.
- Can have **abstract methods** (methods without a body) as well as **concrete methods** (methods with a body).
- Cannot be instantiated directly.
- Supports **inheritance**.
- Can contain **constructors** and **fields** like a regular class.



Static Class in Java

In Java, a **static class** refers to a **static nested class**. Unlike regular classes, a static class is a **class within another class** that is declared as static. It cannot exist independently and is associated with its outer class.

A **static nested class** does not have access to the instance variables or methods of the outer class. However, it can directly access the static members of the outer class.

✅ Why Use a Static Class?

- To logically group classes that are only used within the outer class.
- To increase code encapsulation and reduce complexity.
- To access static members of the outer class without creating an object of the outer class.

java Copy Edit

```java
class OuterClass {
    static String message = "Hello from Outer Class!";

    // Static nested class
    static class StaticNestedClass {
        void displayMessage() {
            System.out.println(message); // Accessing outer class static variable
        }
    }
}

public class Main {
    public static void main(String[] args) {
        // Creating an object of StaticNestedClass without OuterClass object
        OuterClass.StaticNestedClass nestedObj = new OuterClass.StaticNestedClass();
        nestedObj.displayMessage();
    }
}
```

✅ Key Points About Static Classes

1. **No Reference to Outer Class:**
    - A static class **cannot access non-static (instance) members** of the outer class.

2. **Direct Access to Static Members:**
    - It can directly access **static variables and methods** of the outer class.
3. **No Need for Outer Class Object:**
    - You don't need an instance of the outer class to create an object of the static class.
4. **Used for Helper Classes:**
    - Often used for utility classes like Map.Entry, Collections.sort, etc.
5. **Cannot be Top-Level Class:**
    - A top-level class cannot be declared as static in Java.

Final Class in Java

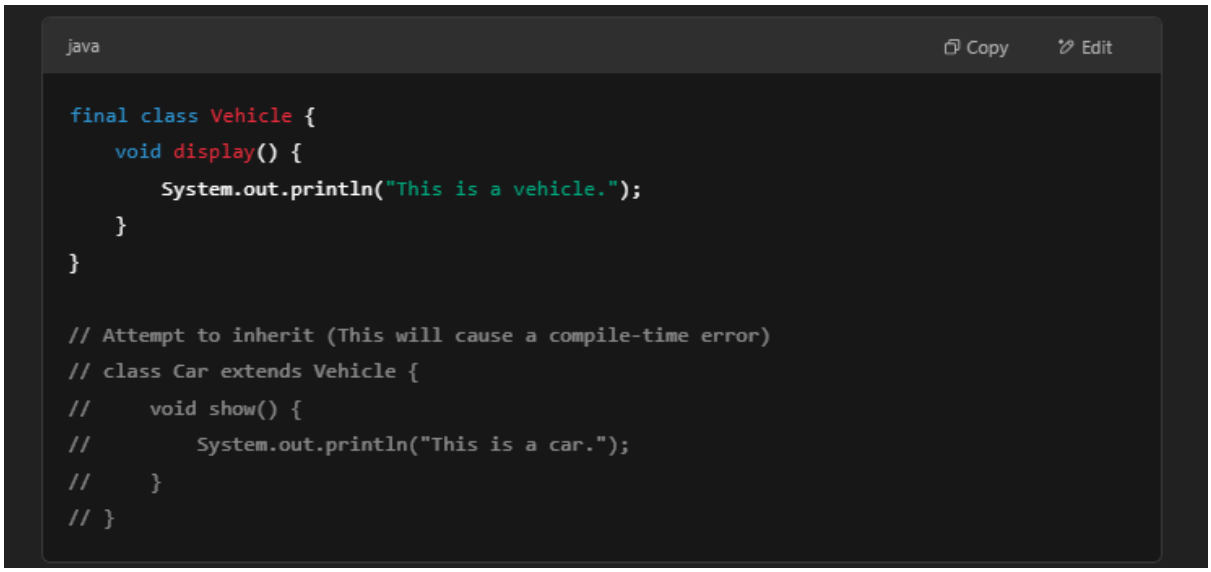
In Java, a **final class** is a class that **cannot be inherited**. This means no other class can extend (subclass) a final class. It is declared using the final keyword.

✅ **Why Use a Final Class?**

- To **prevent inheritance** and ensure the class's behavior remains unchanged.
- Often used for **security purposes** to avoid malicious modifications.
- Used for **immutable classes** like String, Integer, and other wrapper classes.
- A final class can have final and non-final methods and variables.

Commonly Used in Java API:

- Classes like String, Math, and Wrapper classes (Integer, Double) are declared as final.



Constructor Chaining in Java Using this() and super()

Constructor Chaining refers to the process of calling one constructor from another within the same class or in a parent class using this() or super().

- **this()**: Calls another constructor within the same class.
- **super()**: Calls a constructor from the parent (superclass) class.

✅ Using this() for Constructor Chaining (Within the Same Class)

- **this()** is used to call another constructor from the **same class**.
- It helps avoid code duplication by reusing constructors.
- It **must be the first statement** in the constructor.



✅ Using super() for Constructor Chaining (From Parent Class)

- **super()** is used to call the parent class constructor.
- It is **implicitly called** if no constructor is explicitly called in a subclass.
- It **must be the first statement** in the subclass constructor.



Interfaces in Java

In **Java**, an **interface** is a collection of **abstract methods** and **constants**. It provides a way to achieve **abstraction** and **multiple inheritance**. Interfaces are declared using the interface keyword.

✅ **Key Features of Interfaces**

- **Abstract Methods**: By default, all methods in an interface are abstract (without a body) until Java 8.
- **Constants**: All variables in an interface are public, static, and final by default.
- **No Object Creation**: You cannot instantiate an interface directly.
- **Multiple Inheritance**: A class can implement multiple interfaces.
- **Default and Static Methods**: Introduced in Java 8, interfaces can have default and static methods with a body.



✅ Key Differences: Interface vs Abstract Class

Feature	Interface	Abstract Class
Methods	Only abstract methods (until Java 8)	Can have both abstract and non-abstract methods
Multiple Inheritance	Supports multiple inheritance	Supports single inheritance
Constructors	No constructors	Can have constructors
Variables	`public static final` by default	Can have instance variables
Access Modifiers	Methods are implicitly `public`	Can have any access modifier
Method Body	Can have default and static methods (Java 8+)	Abstract methods have no body, non-abstract ones have body

Garbage collection (GC) in Java

- Garbage collection (GC) in Java is an automatic memory management process that reclaims memory occupied by objects that are no longer in use by the application.
- This prevents memory leaks and improves overall program efficiency.

Here's how it works:

1. **Identifying Unused Objects:**
    - **Reachability:** The core concept of Java GC is "reachability."
    - An object is considered "reachable" if there's a live thread that can access it directly or indirectly through a chain of references. Objects that are not reachable are considered garbage.
    - **Root Set:** GC starts with a set of "root" objects, which are always reachable.

These include:

- Local variables in currently executing methods.
- Static variables of loaded classes.
- Active threads.
- JNI (Java Native Interface) references.

2. **Tracing:**

- Starting from the root set, the GC traces through all object references to find all reachable objects.
- Any object not found during this tracing is considered unreachable.

3. **Garbage Collection Algorithms:** Java uses various garbage collection algorithms, each with its own approach to identifying and reclaiming unused memory.

- **Mark and Sweep:**
  - **Mark:** The GC traverses the object graph, marking all reachable objects.
  - **Sweep:** The GC scans the heap and reclaims the memory occupied by unmarked objects.
  - **Drawback:** This can lead to memory fragmentation, as free memory is divided into small, unusable chunks.
- **Mark and Compact:**
  - **Mark:** Similar to Mark and Sweep, reachable objects are marked.
  - **Compact:** After marking, the GC moves all reachable objects to one end of the heap, compacting the memory and eliminating fragmentation.
  - **Cost:** This is more expensive than Mark and Sweep due to the object movement.
- **Copying:**
  - The heap is divided into two halves.
  - Objects are created in one half. When that half is full, the GC copies reachable objects to the other half.
  - The original half is then cleaned, becoming the new allocation space.
  - **Benefit:** Very efficient for short-lived objects.
  - **Drawback:** Requires doubling the heap size.
- **Generational Garbage Collection:**
  - This is the most common approach in modern JVMs.
  - It's based on the observation that most objects have short lifespans.
  - The heap is divided into generations:
    - **Young Generation**
    - **Old Generation**

- **Permanent Generation**

4. **Garbage Collection Collectors in HotSpot JVM:** The HotSpot JVM provides several garbage collectors to choose from, each with different performance characteristics:

- **Serial GC**
- **Parallel GC**
- **Concurrent Mark Sweep (CMS) GC**

5. **Triggering Garbage Collection:** GC is triggered automatically by the JVM when it determines that memory is running low or when certain thresholds are met. You can also request GC using System.gc(), but this is only a hint to the JVM, not a guarantee that GC will run immediately.

Lambda Expressions in Java

- Lambda expressions are a feature introduced in Java 8 that provide a way to write concise and expressive code.
- They are essentially anonymous functions that can be used to implement functional interfaces.
- A functional interface is an interface that contains only one abstract method.

Syntax of Lambda Expression:

(parameters) -> expression

Or

(parameters) -> { statements; }

Components of Lambda Expression:

1. **Parameters:** Similar to method parameters, can be empty or contain multiple parameters.
2. **Arrow Operator (->):** Separates the parameters and the body of the lambda.
3. **Expression or Block of Statements:** The body of the lambda can either be a single expression or a block of code.

Example:

```
// Lambda expression to implement a functional interface
interface MathOperation {
    int operate(int a, int b);
}

public class LambdaExample {
    public static void main(String[] args) {
        // Using lambda expression for addition
        MathOperation addition = (a, b) -> a + b;
        System.out.println("Sum: " + addition.operate(10, 20));
    }
}
```

Generics in Java

- Generics in Java are a powerful feature that allows you to write flexible and reusable code.
- They enable you to define classes, interfaces, and methods with type parameters, which provide type safety at compile time.

Purpose of Generics

- **Type Safety:** Prevents runtime errors by catching type mismatches during compilation.
- **Code Reusability:** Allows you to write generic code that can work with different data types.
- **Elimination of Type Casting:** Reduces the need for explicit type casting.

Syntax of Generics

- Generics use angle brackets (<>) to specify the type parameter.



Types of Generics in Java

1. **Generic Classes:** Classes that accept one or more type parameters.
2. **Generic Interfaces:** Similar to generic classes, interfaces can also use type parameters.
3. **Generic Methods:** Methods that declare their own type parameters.

Wildcards in Generics

Wildcards (?) represent unknown types. They are used for flexibility in method parameters.

Example of Wildcards

```java
public class WildcardExample {
    public static void displayList(List<?> list) {
        for (Object obj : list) {
            System.out.println(obj);
        }
    }

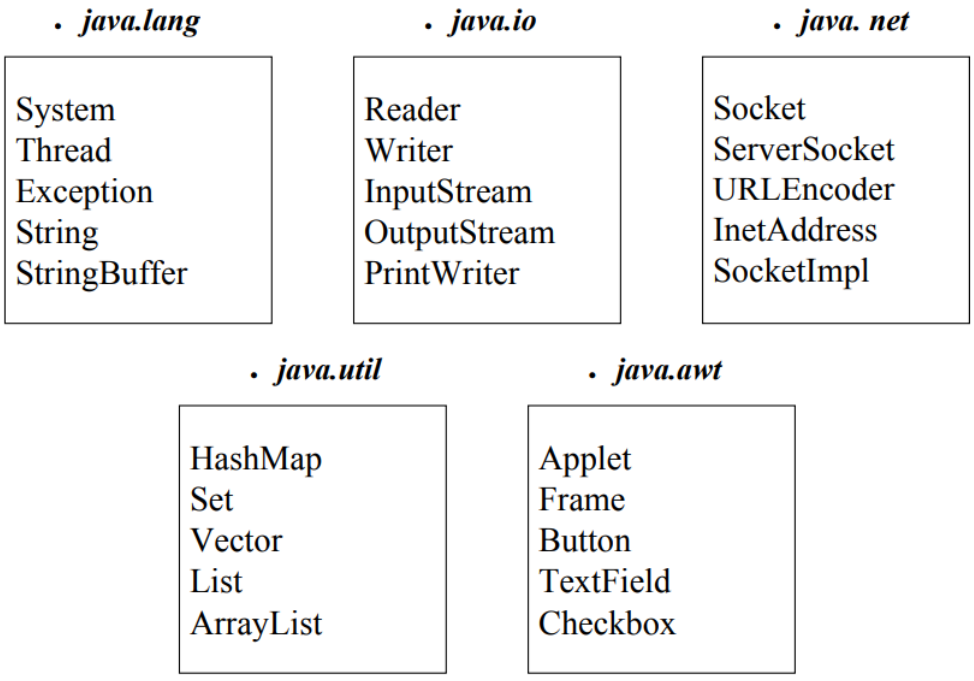
    public static void main(String[] args) {
        List<String> stringList = Arrays.asList("A", "B", "C");
        List<Integer> intList = Arrays.asList(1, 2, 3);

        displayList(stringList);
        displayList(intList);
    }
}
```

Packages in Java

2.2.1 Package Levels

- **Built-in Packages:** Provided by Java (e.g., java.util, java.io)



- **User-defined Packages:** Created by users to organize their own classes.

2.2.2 Creating a Package

- Use the package keyword at the top of the file.
- Example:

```java
package mypackage;
public class MyClass {
    public void display() {
        System.out.println("Hello from MyClass");
    }
}
```

2.2.3 Importing and Using Packages

- Use import t*o bring packages into your code.*
- Example:
- You can also import all classes using import mypackage.*;

```java
import mypackage.MyClass;

public class Main {
    public static void main(String[] args) {
        MyClass obj = new MyClass();
        obj.display();
    }
}
```

2.2.4 Package and Inheritance

- Classes in different packages can be extended using extends if they are declared as public or protected.
- Example:

```java
import mypackage.MyClass;

public class Main {
    public static void main(String[] args) {
        MyClass obj = new MyClass();
        obj.display();
    }
}
```

2.2.5 Package and Access Specifiers

- **Public:** A*ccessible from any package.*

- **Protected:** Accessible within the package and by subclasses.
- **Default (no modifier):** Accessible only within the same package.
- **Private:** Accessible only within the same class.

Packages help in organizing code, avoiding name conflicts, and controlling access using these access specifiers.

Exception Handling in Java

2.3.1 Exception and Error Classes

- **Exception:** Represents issues in the program that can be handled (e.g., IOException, SQLException).
- **Error:** Represents serious problems that cannot be handled by the application (e.g., OutOfMemoryError, StackOverflowError).

2.3.2 Exception Handling

Exception handling in Java is managed using try, catch, and finally blocks.

- **try Block:**
    - The code that may generate an exception is placed inside the try block.
    - It allows you to test a block of code for errors.
    - If an exception occurs, the flow of control is immediately transferred to the corresponding catch block.
- **catch Block:**
    - The catch block contains the code that handles the exception.
    - You can have multiple catch blocks to handle different types of exceptions.
    - The exception object can be accessed within the catch block for detailed error information.
- **finally Block:**
    - The finally block contains code that always executes after the try and catch blocks.
    - It is typically used to release resources like closing files, database connections, or network sockets.
    - It runs regardless of whether an exception occurs or not.

Example of try-catch-finally

```
public class ExceptionExample {
    public static void main(String[] args) {
        try {
            int result = 10 / 0; // This will throw ArithmeticException
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.out.println("Exception caught: " + e.getMessage());
        } finally {
            System.out.println("This block always executes.");
        }
    }
}
```

2.3.3 Throw Statement and Throws Clause

- **throw:** Used to explicitly throw an exception.
- **throws:** Declares exceptions that may occur within a method.
- Example:

```
public void checkNumber(int number) throws IllegalArgumentException {
    if (number < 0) {
        throw new IllegalArgumentException("Number cannot be negative.");
    }
}
```

2.3.4 Custom Exception

- You can create custom exceptions by extending the Exception class.
- Example:

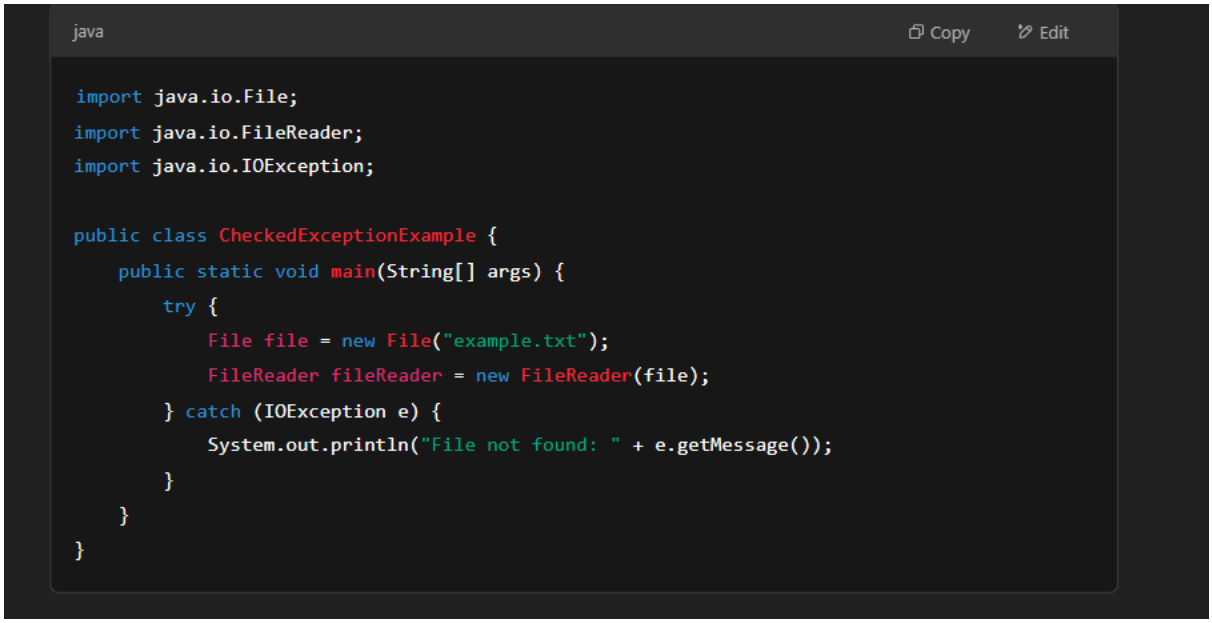
```
class CustomException extends Exception {
    public CustomException(String message) {
        super(message);
    }
}

public class TestCustomException {
    public void validateAge(int age) throws CustomException {
        if (age < 18) {
            throw new CustomException("Age must be 18 or above.");
        } else {
            System.out.println("Valid age.");
        }
    }
}
```

In Java, exceptions are categorized into **Checked** and **Unchecked** exceptions. Here's the difference between them:

1. Checked Exceptions

- **Definition:** Checked exceptions are exceptions that are checked at compile time.
- **Cause:** Usually occur due to external factors like file not found, network issues, or database errors.
- **Handling:** Must be handled using try-catch blocks or declared using the throws keyword.
- **Examples:**
    - IOException
    - SQLException
    - FileNotFoundException
    - ClassNotFoundException



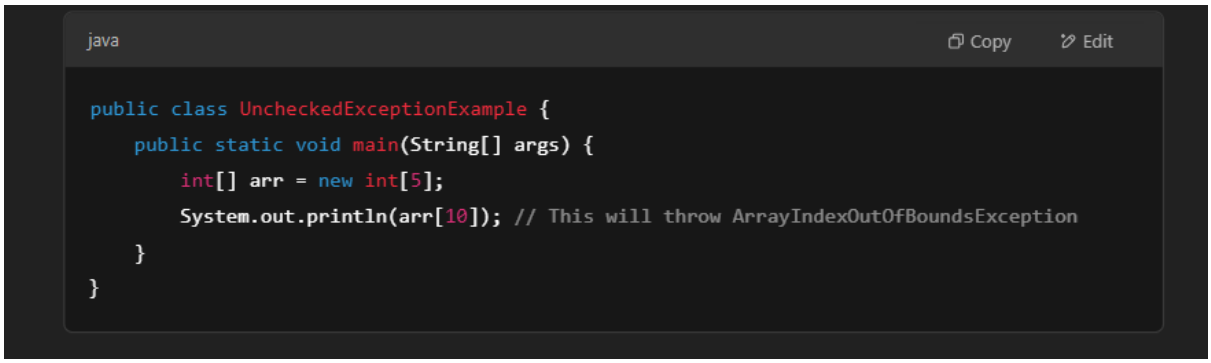
---

2. Unchecked Exceptions

- **Definition:** Unchecked exceptions are exceptions that occur at runtime (not checked during compilation).
- **Cause:** Usually caused by logical errors or faulty programming (e.g., accessing a null object).

- **Handling:** It is not mandatory to handle them using try-catch.
- **Examples:**
    - ArithmeticException
    - NullPointerException
    - ArrayIndexOutOfBoundsException
    - IllegalArgumentException

Example:



Thread in Java

- A **thread** in Java is the smallest unit of a program that can execute **independently**.
- Java supports **multithreading**, allowing multiple threads to run concurrently to improve performance.

Creating a Thread in Java

There are **two ways** to create a thread:

1. **Extending the Thread class**

```java
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running...");
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start(); // Starts the thread
    }
}
```

---

2. **Implementing the Runnable interface**

```java
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Thread is running...");
    }
}

public class Main {
    public static void main(String[] args) {
        Thread t1 = new Thread(new MyRunnable());
        t1.start(); // Starts the thread
    }
}
```

✅ **start() method is used to run a thread** (calling run() directly won't start a new thread).

Thread Methods

Method	Description
`start()`	Starts the thread.
`run()`	Defines the code to be executed.
`sleep(ms)`	Pauses the thread for `ms` milliseconds.
`join()`	Waits for a thread to finish before continuing.
`isAlive()`	Checks if the thread is still running.

```java
class MyThread extends Thread {
    public void run() {
        System.out.println(Thread.currentThread().getName() + " is running");
        try {
            Thread.sleep(1000); // Sleep for 1 second
            System.out.println(Thread.currentThread().getName() + " has finished sleeping");
        } catch (InterruptedException e) {
            System.out.println(e.getMessage());
        }
    }
}

public class ThreadExample {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();

        // Start both threads
        t1.start();
        t2.start();

        // Check if threads are alive
        System.out.println("Is Thread 1 alive? " + t1.isAlive());
        System.out.println("Is Thread 2 alive? " + t2.isAlive());

        try {
            t1.join(); // Wait for t1 to complete
            t2.join(); // Wait for t2 to complete
        } catch (InterruptedException e) {
            System.out.println(e.getMessage());
        }

        // Final check if threads are alive
        System.out.println("Is Thread 1 alive? " + t1.isAlive());
        System.out.println("Is Thread 2 alive? " + t2.isAlive());
        System.out.println("All threads have finished.");
    }
}
```

snappify.com

Thread Life Cycle in Java

In Java, a thread goes through several stages from creation to termination. These stages form the **Thread Life Cycle**.

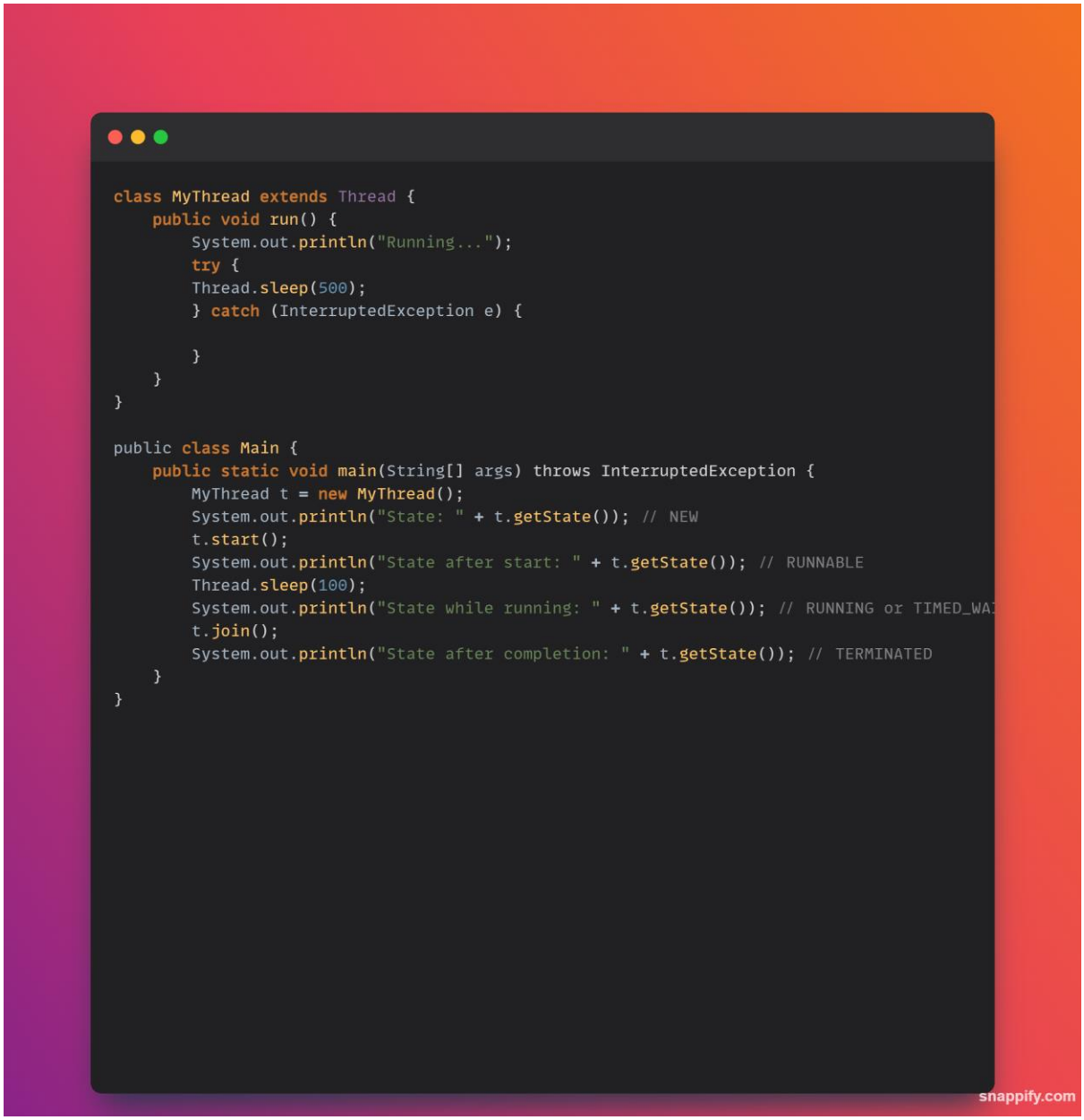
1. Stages of Thread Life Cycle

State	Description
New	Thread is created but not started.
Runnable	Thread is ready to run but waiting for CPU time.
Running	Thread is executing its `run()` method.
Blocked	Thread is waiting for a resource (e.g., lock).
Waiting	Thread is waiting indefinitely for another thread to signal it.
Timed Waiting	Thread waits for a specified time (`sleep()` , `join()` , etc.).
Terminated	Thread has finished execution or was stopped.

2. Java Thread Life Cycle Diagram

rust Copy Edit

```
NEW --> RUNNABLE --> RUNNING --> TERMINATED
                 |
          WAITING / BLOCKED / TIMED WAITING
```



Multithreading in Java

What is Multithreading?

Multithreading is a feature in Java that allows multiple threads to run **concurrently**, making efficient use of CPU.

✅ **Threads share the same memory space but execute independently.**
✅ **Faster execution compared to sequential processing.**
✅ **Improves responsiveness in GUI and server applications.**

Multithreading Example with Two Threads

```
class MyThread extends Thread {
    public void run() {
        for (int i = 1; i ⩽ 5; i++) {
            System.out.println(Thread.currentThread().getName() + " - " + i);
            try { Thread.sleep(500); } catch (InterruptedException e) {}
        }
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();

        t1.start();
        t2.start();
    }S
}
```

Types of multithreading :

User Threads in Java:

A **User Thread** is a thread created explicitly by the programmer to perform a task **independently** from the main program. These threads run **until their execution is complete**, unless the Java Virtual Machine (JVM) is terminated.

✅ **By default, all threads in Java are user threads unless explicitly marked as daemon threads.**

Characteristics of User Threads

✓ **Independent Execution** → Runs separately from the main thread.
✓ **Not Automatically Terminated** → JVM waits for all user threads to finish before exiting.

✓ **Can be Created Explicitly** → Using Thread class or Runnable interface.

✓ **Higher Priority than Daemon Threads** → Ensures important tasks are completed.

How to Create a User Thread?

1. Extending the Thread Class

```
class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("User Thread: " + i);
            try { Thread.sleep(1000); } catch (InterruptedException e) {}
        }
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start(); // Starts a user thread
    }
}
```

✅ **Output:** The user thread runs independently and prints numbers **1 to 5**.

---

2. Implementing the Runnable Interface

```
class MyRunnable implements Runnable {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Runnable Thread: " + i);
            try { Thread.sleep(1000); } catch (InterruptedException e) {}
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Thread t1 = new Thread(new MyRunnable());
        t1.start(); // Starts a user thread
    }
}
```

✅ **Runnable is preferred** over Thread because it allows multiple inheritance.

Key Points to Remember

✓ User threads execute independently and must finish before JVM exits.
✓ Created using Thread class or Runnable interface.
✓ JVM does NOT terminate until all user threads complete.
✓ Daemon threads stop automatically when user threads finish execution.

Daemon thread:

A **daemon thread** in Java is a background thread that runs to support other threads. It is typically used for tasks like garbage collection, monitoring, or background maintenance.

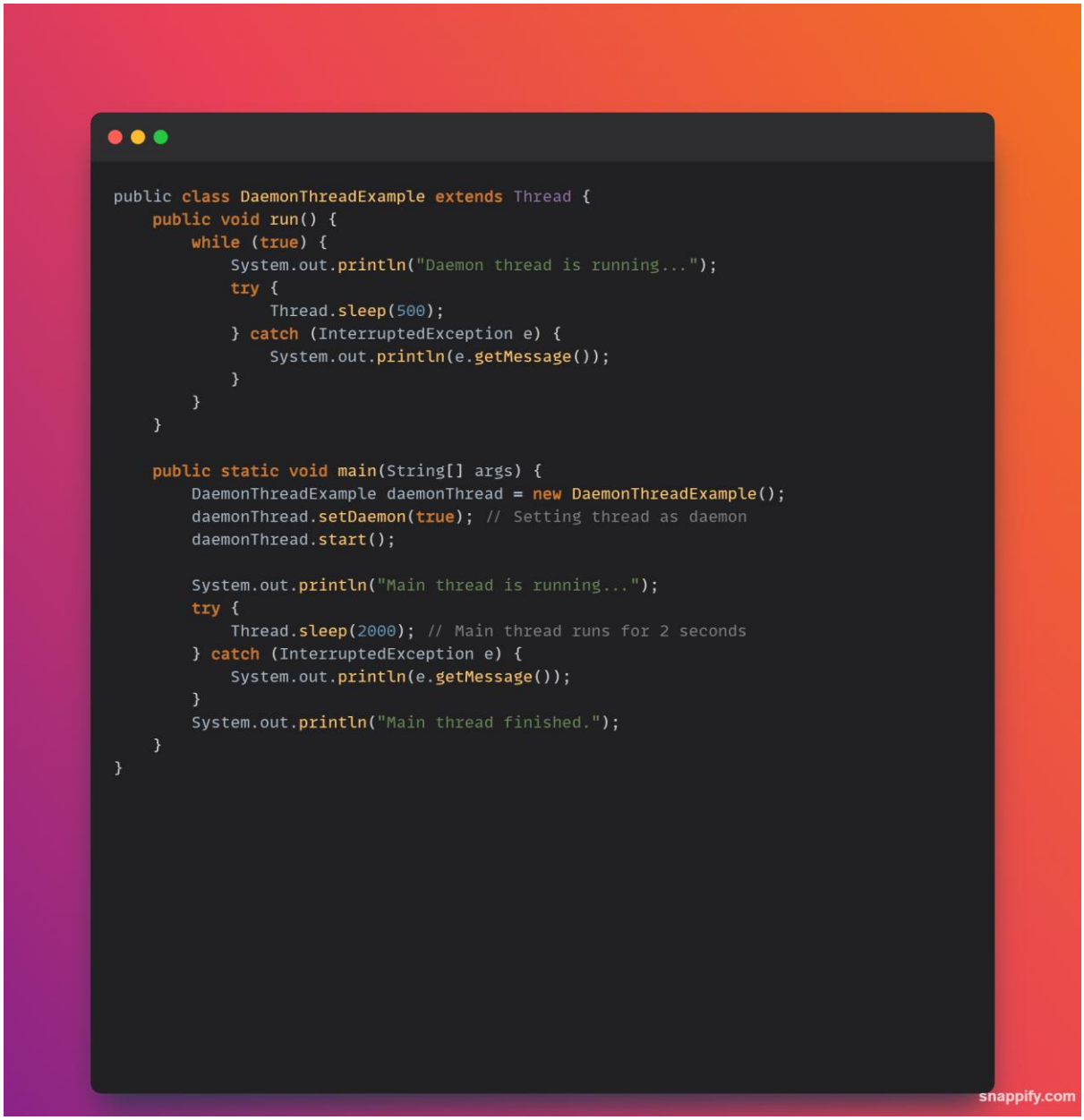
Key Characteristics of Daemon Threads

- **Low Priority**: They usually run in the background with lower priority compared to user threads.
- **Ends with User Threads**: If all user threads finish, JVM automatically terminates daemon threads.
- **Not for Critical Tasks**: Since they can be terminated suddenly, they are not suitable for critical operations.

Example of Daemon Thread in Java

Here's how you can create and use a daemon thread:

1. **Create a Thread**: Define a thread using the Thread class or Runnable interface.
2. **Set as Daemon**: Use thread.setDaemon(true) to mark it as a daemon thread.
3. **Start the Thread**: Call start() to begin its execution.



Synchronization in Java

Synchronization in Java is a mechanism that ensures **only one thread** can access a shared resource at a time. It helps prevent **race conditions** and **data inconsistency** when multiple threads work on the same resource.

✅ **Prevents multiple threads from modifying a shared resource simultaneously.**
✅ **Ensures thread safety in multithreaded programs.**
✅ **Uses synchronized keyword to lock methods or code blocks.**

The Problem Without Synchronization (Race Condition)

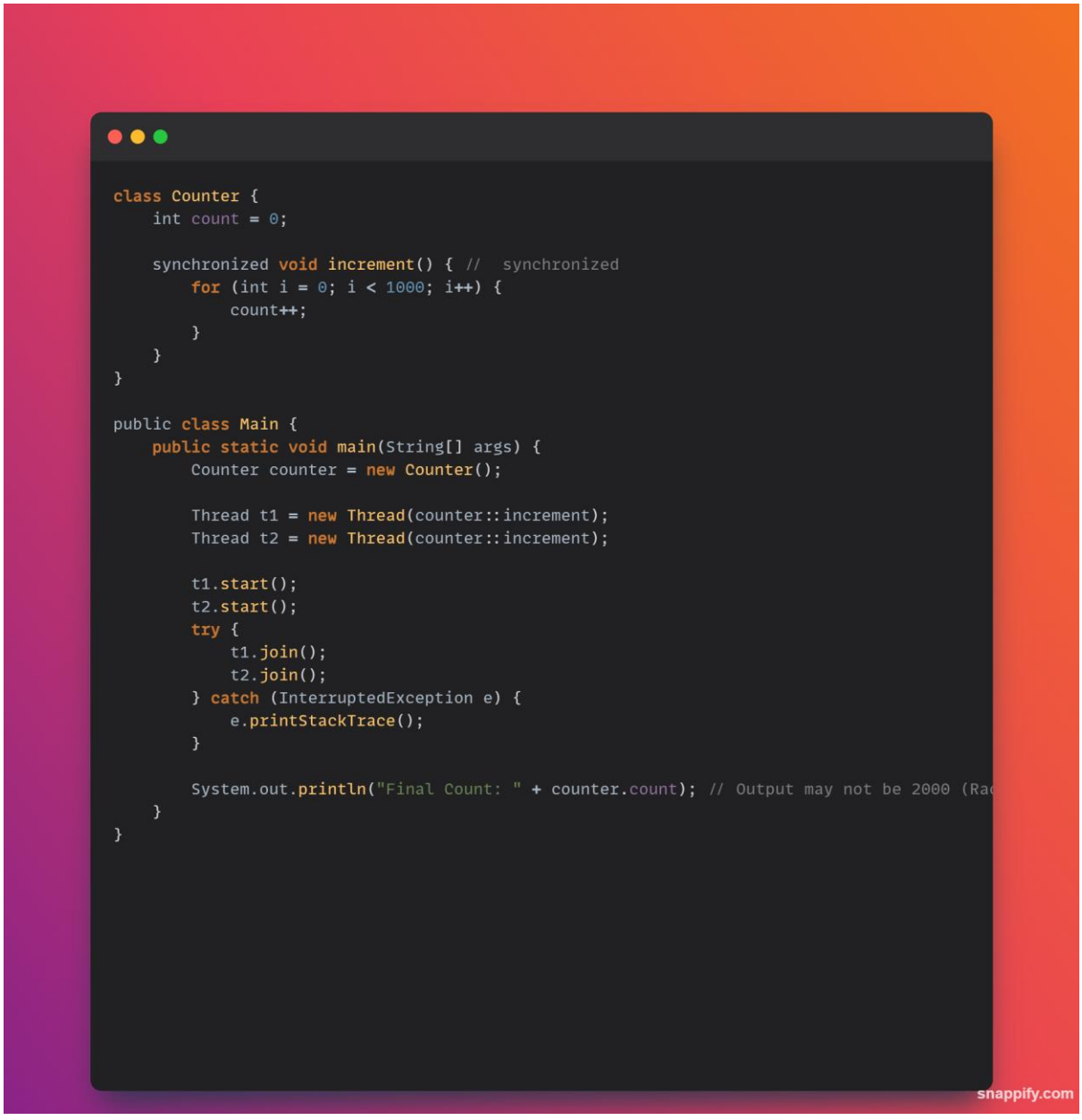
When multiple threads try to modify a shared resource **simultaneously**, it can lead to inconsistent data.



✅ **Problem:** The output may vary (not always 2000) because multiple threads modify count **at the same time**.

Using Synchronization to Fix the Issue

Java provides the synchronized keyword to ensure **one thread at a time** can execute a critical section.



Types:

Block Synchronization (More Efficient)

Instead of synchronizing the entire method, we can synchronize only the **critical section**.

```
class Counter {
    int count = 0;

    void increment() {
        synchronized (this) { // Locks only this block
            count++;
        }
    }
}
```

Static Synchronization (For Static Methods)

If multiple threads access a **static method**, we use synchronized on the **class level**.

```
class Counter {
    static int count = 0;

    synchronized static void increment() { // Locks at the class level
        count++;
    }
}
```

✅ **Ensures thread safety for static variables.**

Using Lock for Advanced Synchronization

Instead of synchronized, Java provides Lock (from java.util.concurrent.locks), which is **more flexible**.

```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

class Counter {
    int count = 0;
    private Lock lock = new ReentrantLock();

    void increment() {
        lock.lock(); // Locking
        try {
            count++;
        } finally {
            lock.unlock(); // Unlocking
        }
    }
}
```

✅ **Allows manual locking and unlocking, preventing deadlocks**

Deadlock in Java

A **deadlock** occurs when **two or more threads are waiting for each other to release resources**, and neither thread can proceed. This results in a **permanent blocking state** where the program hangs.

✅ **Deadlocks happen when multiple threads hold locks on shared resources and wait for each other.**
✅ **Common in multithreading when improper locking mechanisms are used.**



How to Avoid Deadlocks?

✓ 1. Use Lock Ordering (Acquire Locks in Same Order)

✓ 2. Use tryLock() (Avoid Waiting Forever)

✓ 3. Use a Timeout for Lock Acquisition

Inter-Thread Communication?

Inter-thread communication in Java allows multiple threads to coordinate with each other while working on shared resources.

🔷 **wait()** → Tells the thread to **release the lock and wait** until another thread notifies it.
🔷 **notify()** → Wakes up **one waiting thread** randomly.
🔷 **notifyAll()** → Wakes up **all waiting threads** on the same object.

Simple Explanation

- **Thread T1** calls wait() → It pauses and releases the lock.
- **Thread T2** does some work and calls notify() → This wakes up T1.
- **notifyAll()** can be used if multiple threads are waiting, waking them all up.

Why Use wait() and notify()?

Imagine a **Producer-Consumer Problem** where:
✅ The **Producer** produces items and adds them to a queue.
✅ The **Consumer** takes items from the queue and processes them.
✅ If the queue is **empty**, the Consumer should wait.
✅ If the queue is **full**, the Producer should wait.

💡 **Important:**
✅ wait(), notify(), and notifyAll() must be called inside a **synchronized** block or method.
✅ A thread must own the lock before calling these methods.

```java
import java.util.LinkedList;
import java.util.Queue;

class SharedQueue {
    private final int capacity = 5; // Maximum size of the queue
    private final Queue<Integer> queue = new LinkedList<>();

    public synchronized void produce(int value) throws InterruptedException {
        while (queue.size() == capacity) {
            System.out.println("Queue is full. Producer is waiting...");
            wait();
        }
        queue.add(value);
        System.out.println("Produced: " + value);
        notify(); // Notify Consumer
    }

    public synchronized void consume() throws InterruptedException {
        while (queue.isEmpty()) {
            System.out.println("Queue is empty. Consumer is waiting...");
            wait();
        }
        int value = queue.poll();
        System.out.println("Consumed: " + value);
        notify(); // Notify Producer
    }
}

public class ProducerConsumerExample {
    public static void main(String[] args) {
        SharedQueue sharedQueue = new SharedQueue();

        // Producer thread
        Thread producer = new Thread(() -> {
            for (int i = 1; i <= 10; i++) {
                try {
                    sharedQueue.produce(i);
                    Thread.sleep(500); // Simulating time to produce
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        });

        // Consumer thread
        Thread consumer = new Thread(() -> {
            for (int i = 1; i <= 10; i++) {
                try {
                    sharedQueue.consume();
                    Thread.sleep(1000); // Simulating time to consume
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        });

        producer.start();
        consumer.start();
    }
}
```

snappify.com

Fork/Join Framework in Java

What is Fork/Join Framework?

The **Fork/Join framework** in Java **divides a big task into smaller subtasks** (forking) and then combines their results (joining).
It is **designed for parallel execution** using multiple processor cores.

✅ Introduced in Java 7 (java.util.concurrent package)
✅ Works best for recursive tasks (Divide and Conquer approach)
✅ Uses ForkJoinPool to manage worker threads
✅ Uses RecursiveTask<V> (returns a result) or RecursiveAction (no result)

Explanation of Key Concepts

🔹 **RecursiveTask<V>** → Used when a result is needed.
🔹 **ForkJoinPool** → Manages worker threads for parallel execution.
🔹 **fork()** → Starts a subtask asynchronously in another thread.
🔹 **join()** → Waits for a subtask to finish and retrieves its result.
🔹 **compute()** → Defines how the task is divided and solved.

What This Covers:

✅ **ForkJoinPool** → Executes the task
✅ **RecursiveTask** → Returns a result
✅ **compute()** → Performs a simple addition

```
import java.util.concurrent.*;

class MyTask extends RecursiveTask<Integer> {
    protected Integer compute() { return 10 + 20; } // Simple computation
}

public class ForkJoinSmall {
    public static void main(String[] args) {
        ForkJoinPool pool = new ForkJoinPool();
        System.out.println(pool.invoke(new MyTask())); // Output: 30
    }
}
```

Asynchronous Processing in Java

What is Asynchronous Processing?

Asynchronous processing allows tasks to run **independently** without blocking the main thread.

✅ Improves performance in **multi-threaded** applications.

✅ Uses **CompletableFuture, ExecutorService, Threads**, etc.

```
short Example Using CompletableFuture

import java.util.concurrent.CompletableFuture;

public class AsyncExample {
    public static void main(String[] args) {
        CompletableFuture.runAsync(() → System.out.println("Task running asynchronously!"));
        System.out.println("Main thread continues...");
    }
}
```

What This Covers:

✅ **runAsync()** → Runs a task in the background.

✅ **Non-blocking execution** → Main thread is not stopped.

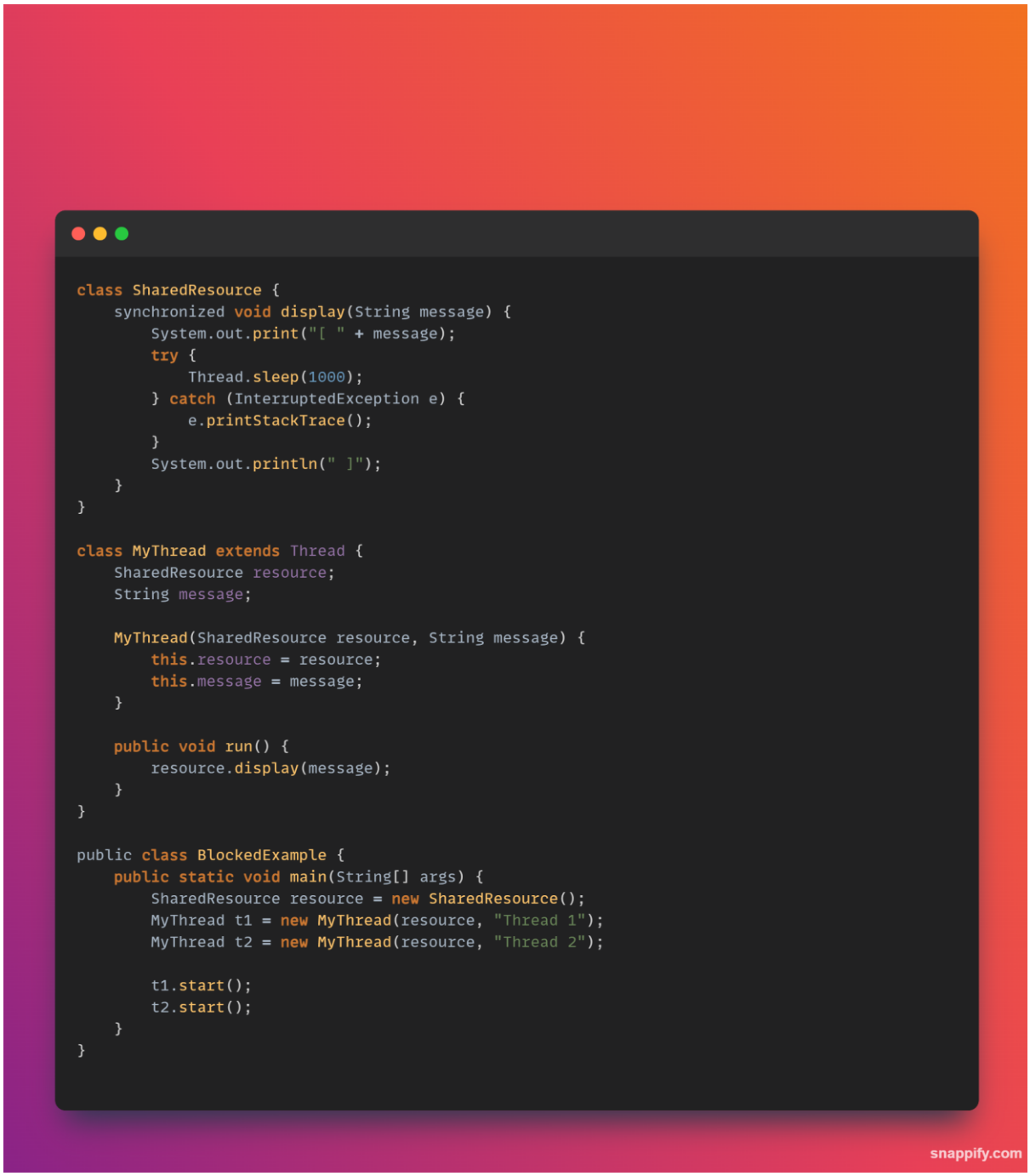
1. Blocked Thread

A **Blocked Thread** is a thread that is **waiting** to gain access to a resource that is currently being used by another thread.

- It happens when a thread tries to enter a **synchronized block** or **method** and another thread is already holding the lock.
- The thread remains in the **Blocked state** until the lock is released.

📌 **Example Scenario for Blocked Thread**

- Two threads trying to access a synchronized method on the same object.

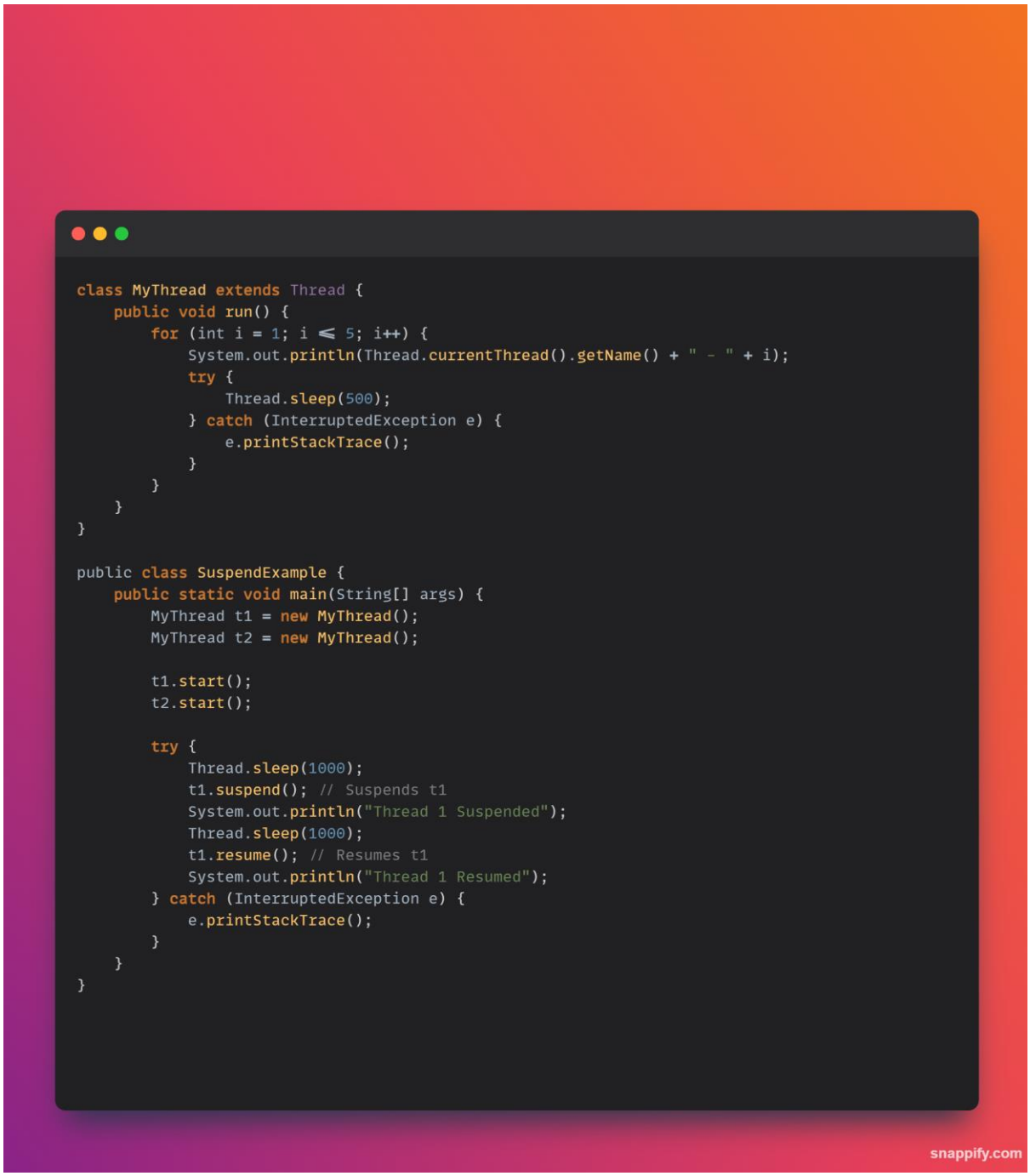


🔎 2. Suspended Thread

A **Suspended Thread** is a thread that is **paused** using the suspend() method and can be resumed later using the resume() method.

- It is an **outdated** method and is **not recommended** because it can lead to **deadlocks**.

- The thread remains in the **Suspended state** until it is explicitly resumed.



Files and directories in java i/o

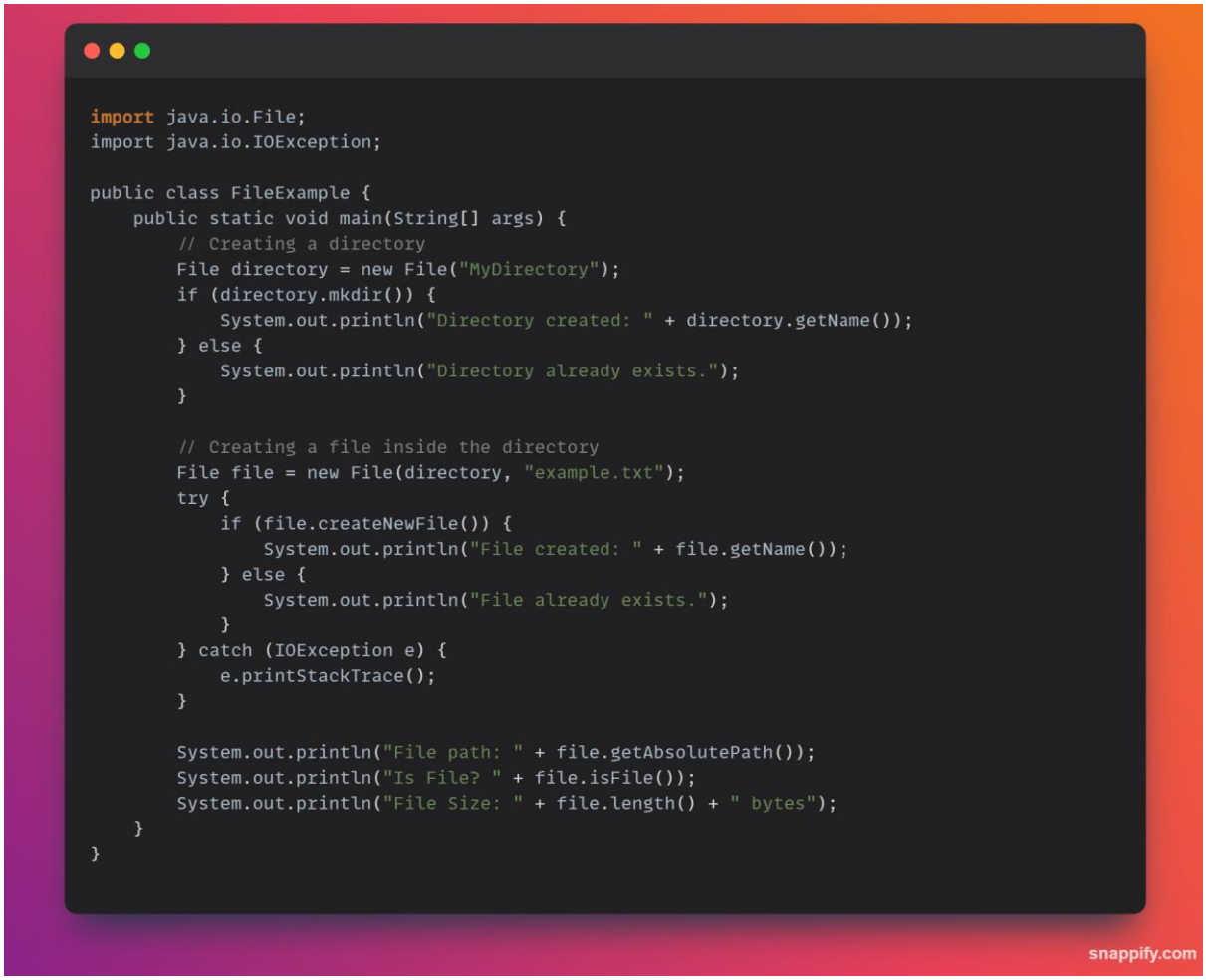
In Java, the **java.io** package provides classes for file and directory manipulation. Here's an overview of the most commonly used classes and their purposes:

🗁 File Class

- Located in java.io.File
- Represents both **files** and **directories**
- Provides methods to create, delete, and get information about files or directories.

Key Methods

- boolean createNewFile() - Creates a new file
- boolean mkdir() - Creates a single directory
- boolean delete() - Deletes a file or directory
- boolean exists() - Checks if the file or directory exists
- boolean isFile() - Checks if it is a file
- boolean isDirectory() - Checks if it is a directory
- String[] list() - Lists the names of files and directories in a directory
- long length() - Returns the length of the file in bytes



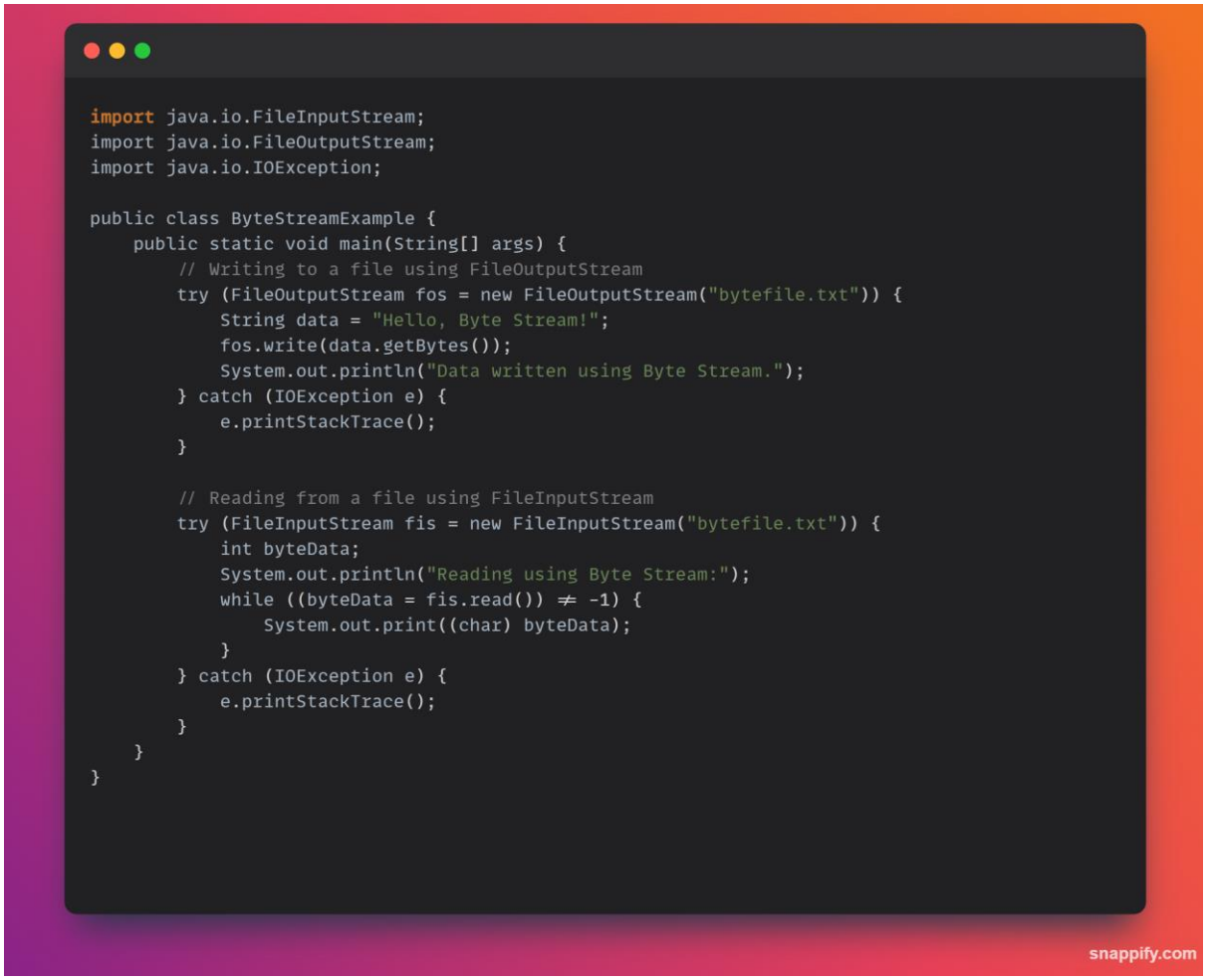
Byte and Character Streams

In Java, **streams** are used to perform input and output operations. Java provides two main types of streams:

1. **Byte Streams** – For handling binary data (e.g., images, videos, audio, etc.).
2. **Character Streams** – For handling character data (e.g., text files).

📦 1. Byte Streams

- **Used for**: Binary data (e.g., images, audio files, video files).
- **Classes**: FileInputStream, FileOutputStream
- **Data Representation**: Reads and writes data in **bytes** (8 bits).

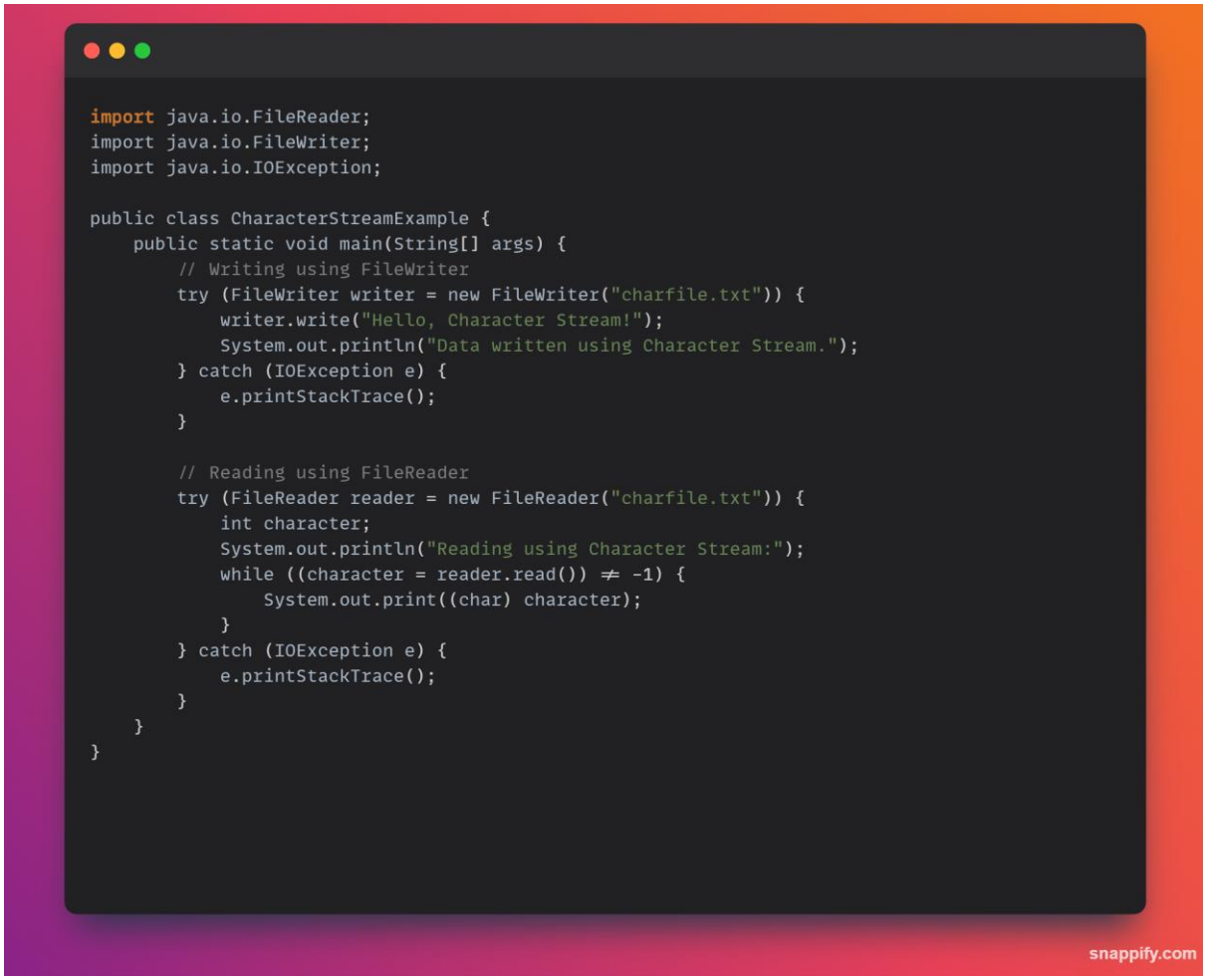


Explanation:

- FileOutputStream → Writes data as bytes using .write().
- FileInputStream → Reads data byte by byte using .read().

✨ 2. Character Streams

- **Used for**: Character data (e.g., text files, XML files).
- **Classes**: FileReader, FileWriter
- **Data Representation**: Reads and writes data in **characters** (16 bits, supports Unicode).



Explanation:

- FileWriter → Writes data as characters using .write().
- FileReader → Reads data character by character using .read().

PrintWriter Class in Java

📜 PrintWriter Class in Java

The PrintWriter class in Java, part of the java.io package, is used to print formatted representations of objects to a text output stream. It's often used for writing character data to files or console output.

✅ Key Features of PrintWriter

- Allows writing **text data** to files or other output streams.
- Supports **formatted text** similar to System.out.printf().
- Can write directly using print(), println(), or printf().
- Provides options for **automatic flushing** when writing to files or console.

🛠️ Creating a PrintWriter Object

You can create a PrintWriter using several constructors:

1. **Writing to a File**:

```java
PrintWriter writer = new PrintWriter("output.txt");
```

2. **Using File Object**:

```java
File file = new File("example.txt");
PrintWriter writer = new PrintWriter(file);
```

3. **Writing to Console**:

```java
PrintWriter writer = new PrintWriter(System.out);
```

Example 1: Writing to a File

- **writer.println() → Prints data with a newline.**
- **writer.print() → Prints data without a newline.**
- **writer.printf() → Prints formatted data, similar to System.out.printf().**
- **The use of try-with-resources ensures automatic resource management.**

```java
import java.io.PrintWriter;
import java.io.IOException;

public class PrintWriterExample {
    public static void main(String[] args) {
        try (PrintWriter writer = new PrintWriter("printwriter_example.txt")) {
            writer.println("Hello, PrintWriter!");
            writer.println(123);
            writer.println(45.67);
            writer.printf("Formatted number: %.2f%n", 12.34567);
            System.out.println("Data written successfully.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Example 2: Writing to Console

- new PrintWriter(System.out, true) → true enables **auto-flushing**.
- consoleWriter.printf() → Formats and prints data to the console.

```java
import java.io.PrintWriter;

public class ConsoleExample {
    public static void main(String[] args) {
        PrintWriter consoleWriter = new PrintWriter(System.out, true);
        consoleWriter.println("This is printed using PrintWriter.");
        consoleWriter.printf("Formatted output: %d, %.2f, %s%n", 25, 3.14159, "Java");
    }
}
```

Advantages of PrintWriter

- Supports easy and readable output with println(), print(), and printf().
- Provides **formatted output** with built-in formatting options.
- Handles character encoding by wrapping other streams.

Unlike BufferedWriter, PrintWriter does **not throw IOExceptions** directly; instead, you should check checkError() for any errors.

FileReader and FileWriter

- Located in java.io.FileReader and java.io.FileWriter
- Used for reading and writing character data.
- Best suited for text files.

Example: Writing to a File

```java
import java.io.FileWriter;
import java.io.IOException;

public class FileWriterExample {
    public static void main(String[] args) {
        try (FileWriter writer = new FileWriter("output.txt")) {
            writer.write("Hello, Java I/O!");
            System.out.println("Successfully written to the file.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

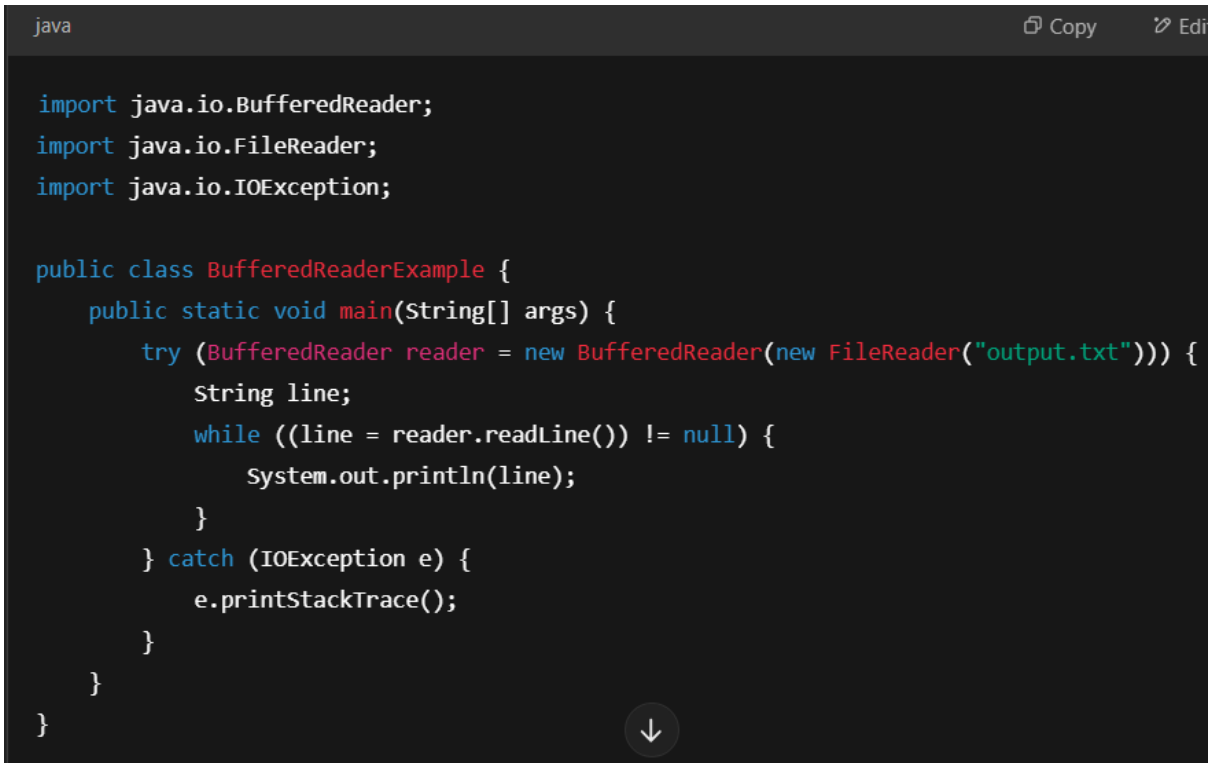
Example: Reading from a File

```java
import java.io.FileReader;
import java.io.IOException;

public class FileReaderExample {
    public static void main(String[] args) {
        try (FileReader reader = new FileReader("output.txt")) {
            int character;
            while ((character = reader.read()) != -1) {
                System.out.print((char) character);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

BufferedReader and BufferedWriter

- Located in java.io.BufferedReader and java.io.BufferedWriter
- Provides efficient reading and writing using buffers for large files.



📖 Random Access Files in Java

- Java provides the RandomAccessFile class, which allows reading and writing data at any position within a file.
- Unlike other file I/O classes that read or write data sequentially,
- RandomAccessFile supports **non-sequential (random)** access, making it ideal for applications like databases, log files, or media files.

🔑 Key Features of RandomAccessFile

- Supports both **reading** and **writing** using the same object.
- Allows file navigation using **seek()** to move the file pointer to a specific location.
- Useful for modifying files without overwriting everything.
- Provides methods for reading and writing primitive data types (int, double, char, etc.).

🛠️ Creating a RandomAccessFile Object

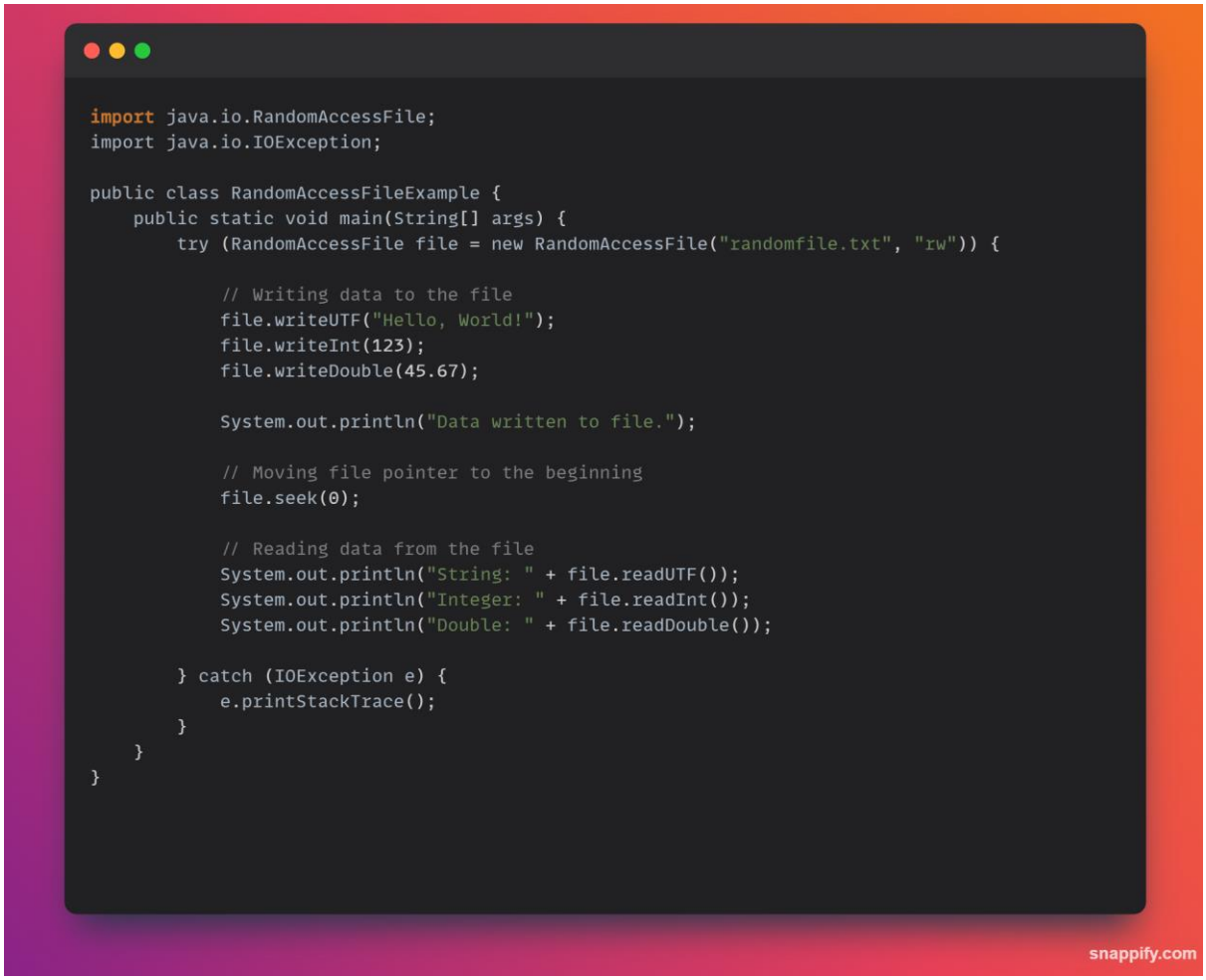
The RandomAccessFile constructor takes two parameters:

1. **File**: The path of the file.
2. **Mode**: The mode in which the file is opened.
    - "r" → Read-only mode
    - "rw" → Read and write mode
    - "rws" → Read, write, and sync data to storage
    - "rwd" → Read, write, and sync only data (not metadata)

```java
RandomAccessFile file = new RandomAccessFile("example.txt", "rw");
```

🧩 Common Methods in RandomAccessFile

Method	Description
`writeInt(int i)`	Writes an integer to the file.
`writeUTF(String s)`	Writes a UTF-encoded string.
`readInt()`	Reads an integer from the file.
`readUTF()`	Reads a UTF-encoded string.
`seek(long position)`	Moves the file pointer to a specified position.
`getFilePointer()`	Returns the current position of the file pointer.
`length()`	Returns the length of the file in bytes.
`close()`	Closes the file.



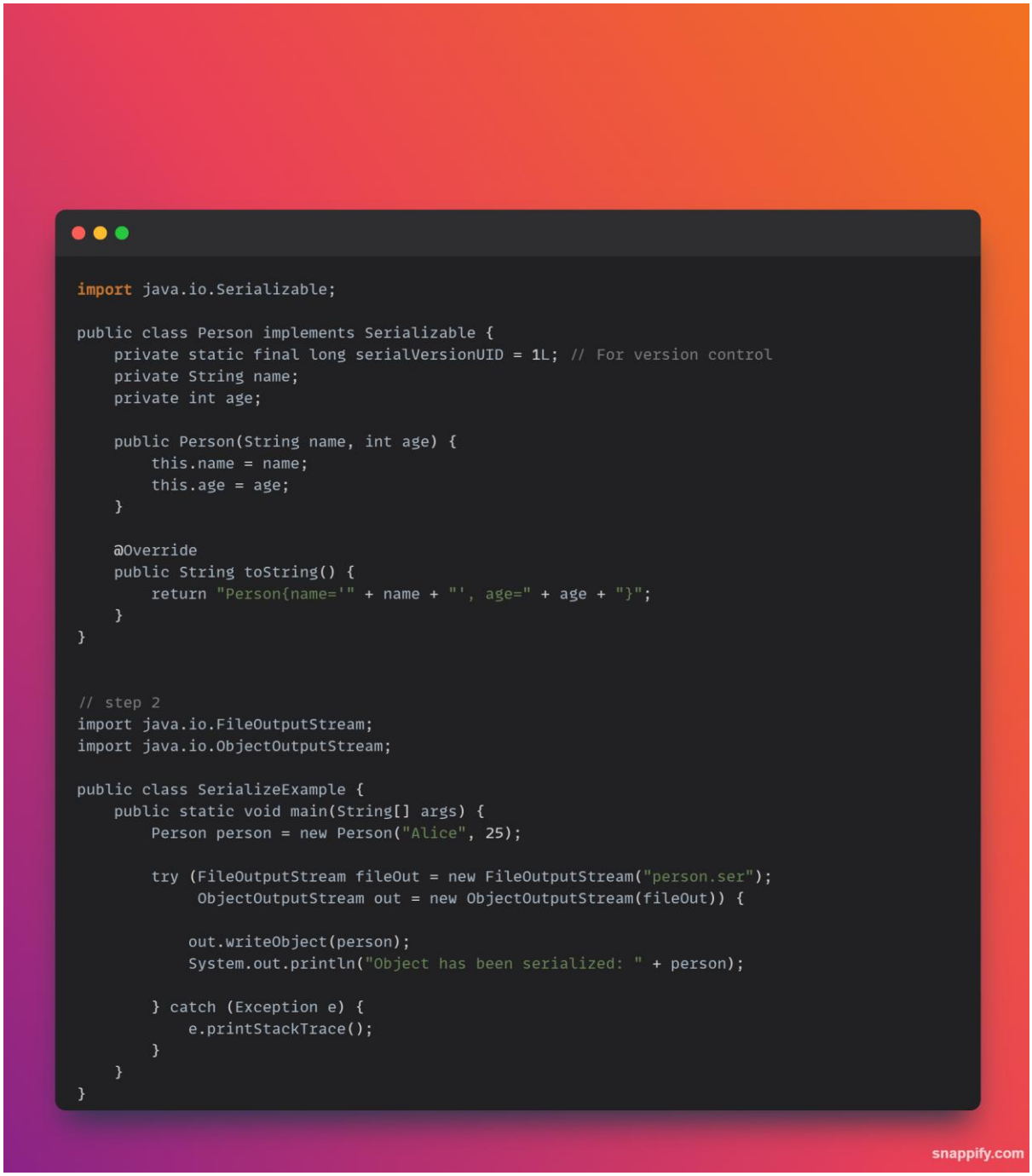
📄 Serialization and Deserialization in Java

Serialization and **Deserialization** are mechanisms in Java used to convert objects into a format that can be easily stored or transmitted and then reconstructed later.

- **Serialization** → Converting a Java object into a stream of bytes (binary format) for storage or transmission.
- **Deserialization** → Converting the byte stream back into a Java object.

✅ Steps to Serialize an Object

1. Implement the Serializable interface.
2. Use ObjectOutputStream to write objects to a file.
3. Use FileOutputStream to connect the object stream to a file.



```java
import java.io.Serializable;

public class Person implements Serializable {
    private static final long serialVersionUID = 1L; // For version control
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public String toString() {
        return "Person{name='" + name + "', age=" + age + "}";
    }
}


// step 2
import java.io.FileOutputStream;
import java.io.ObjectOutputStream;

public class SerializeExample {
    public static void main(String[] args) {
        Person person = new Person("Alice", 25);

        try (FileOutputStream fileOut = new FileOutputStream("person.ser");
             ObjectOutputStream out = new ObjectOutputStream(fileOut)) {

            out.writeObject(person);
            System.out.println("Object has been serialized: " + person);

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

snappify.com

✅ Steps to Deserialize an Object

1. Use ObjectInputStream to read objects from a file.
2. Use FileInputStream to connect the object stream to a file.

```java
import java.io.Serializable;

public class Person implements Serializable {
    private static final long serialVersionUID = 1L; // For version control
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public String toString() {
        return "Person{name='" + name + "', age=" + age + "}";
    }
}


// step 2
import java.io.FileInputStream;
import java.io.ObjectInputStream;

public class DeserializeExample {
    public static void main(String[] args) {
        try (FileInputStream fileIn = new FileInputStream("person.ser");
             ObjectInputStream in = new ObjectInputStream(fileIn)) {

            Person person = (Person) in.readObject();
            System.out.println("Object has been deserialized: " + person);

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

snappify.com

Collection in java

In **Java**, a **Collection** is a framework that provides an architecture to store and manipulate a group of objects. It is a part of the **Java Collections Framework (JCF)** which includes interfaces, classes, and algorithms to manage data efficiently.

Key Interfaces in Collections

1. **Collection Interface**: The root interface of the collection hierarchy.
2. **List Interface**: An ordered collection (e.g., ArrayList, LinkedList).
3. **Set Interface**: A collection that does not allow duplicate elements (e.g., HashSet, LinkedHashSet, TreeSet).
4. **Queue Interface**: A collection used for holding elements before processing, typically in FIFO (e.g., PriorityQueue, LinkedList).
5. **Map Interface**: Not a true collection, but part of the framework. It stores key-value pairs (e.g., HashMap, TreeMap, LinkedHashMap).

Advantages of Collections in Java

- **Dynamic Sizing**: Collections grow and shrink dynamically compared to arrays.
- **Convenient Methods**: Provide built-in methods for searching, sorting, and manipulating data.
- **Efficient Performance**: Data structures like HashMap and LinkedList are optimized for specific operations.

Stream in java:

- **Streams** in Java are a powerful feature introduced in **Java 8** as part of the **java.util.stream** package.
- They provide a functional and declarative way to process collections of data efficiently.

Streams allow operations like:

- Filtering
- Mapping
- Sorting
- Reducing
- Collecting

They are often used to write clean, concise, and readable code, especially when dealing with large datasets.

✅ **Key Characteristics of Streams**

1. **Functional**: Uses functional programming principles with lambda expressions.
2. **Lazy Evaluation**: Intermediate operations are not executed until a terminal operation is invoked.
3. **Parallel Processing**: Can be executed in parallel using .parallelStream() for faster results.
4. **Immutable**: Streams do not modify the original data; they create new streams with the results.

Creating a Stream

You can create a stream from:

- **Collections**: list.stream()
- **Arrays**: Arrays.stream(array)
- **Stream.of()**: Stream.of(1, 2, 3)
- **Files**: Files.lines(Path)
- **Generate or Iterate**: Stream.generate() or Stream.iterate()

1. Intermediate Operations

These return a new stream. They are lazy and executed only when a terminal operation is called. Examples include:

- filter()
- map()
- sorted()
- distinct()
- limit()

2. Terminal Operations

These return a result or a side effect and end the stream. Examples include:

- collect()
- forEach()
- reduce()
- count()
- min() / max()
- anyMatch(), allMatch(), noneMatch()

```java
import java.util.*;
import java.util.stream.Collectors;

public class StreamExample {
    public static void main(String[] args) {
        List<String> names =new ArrayList<>();
        names.add("abc");
        names.add("j1");

        List<String> filteredNames = names.stream()
            .filter(name -> name.startsWith("j"))
            .collect(Collectors.toList());

        System.out.println(filteredNames);
    }
}
```

```java
import java.util.*;
import java.util.stream.Collectors;

public class MapExample {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(5, 2, 8, 3, 1);

        List<Integer> squaredSortedNumbers = numbers.stream()
            .map(num -> num * num)
            .sorted()
            .collect(Collectors.toList());

        System.out.println(squaredSortedNumbers); // Output: [1, 4, 9, 25, 64]
    }
}
```

Set in Java

A **Set** in Java is a part of the **Java Collections Framework** and represents a collection of unique elements. It does **not allow duplicates** and is useful for tasks where uniqueness is required.

Key Features of Set

- No duplicates allowed.
- Allows one null value (except in TreeSet).
- Unordered unless using TreeSet (which is sorted).
- Backed by different implementations like **HashSet** and **TreeSet**.

1.HashSet

✅ Overview

HashSet is a part of the **Java Collections Framework** and is implemented using a **Hash Table**. It is a collection that stores elements in an **unordered** manner and ensures **unique** elements by using **hashing**.

HashSet is defined in the java.util package and implements the Set interface.

📌 Key Characteristics of HashSet

- **No Duplicates**: HashSet does not allow duplicate values.
- **Unordered Collection**: It does **not** maintain the order of elements.
- **Null Allowed**: It allows one null value.
- **Fast Operations**: Provides constant time (O(1)) complexity for add(), remove(), and contains().
- **Backed by HashMap**: Internally, HashSet uses a HashMap to store data.

```java
import java.util.HashSet;

public class HashSetExample {
    public static void main(String[] args) {
        HashSet<String> fruits = new HashSet<>();

        // Adding elements
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Cherry");
        fruits.add("Banana"); // Duplicate - will not be added
        fruits.add(null); // Null allowed

        System.out.println("HashSet: " + fruits);

        // Check if element exists
        System.out.println("Contains 'Apple': " + fruits.contains("Apple"));

        // Remove an element
        fruits.remove("Cherry");
        System.out.println("After Removal: " + fruits);

        // Size of HashSet
        System.out.println("Size of HashSet: " + fruits.size());
    }
}
```

🛠️ Common Methods in HashSet

Method	Description
`add(E e)`	Adds an element to the set if not already present.
`remove(Object o)`	Removes the specified element from the set.
`contains(Object o)`	Returns `true` if the set contains the element.
`size()`	Returns the number of elements in the set.
`isEmpty()`	Returns `true` if the set is empty.
`clear()`	Removes all elements from the set.
`iterator()`	Returns an iterator to traverse through the set.
`toArray()`	Converts the set into an array.

2.TreeSet

✅ Overview

TreeSet in Java is a part of the **Java Collections Framework** and implements the **Set** interface. It is a collection that stores elements in a **sorted** and **unique** manner. Internally, it uses a **Red-Black Tree** (a self-balancing binary search tree) to store elements.

TreeSet is defined in the java.util package and implements the NavigableSet and SortedSet interfaces.

📌 Key Characteristics of TreeSet

- **Sorted Order**: Elements are maintained in **ascending** order (natural order or using a custom comparator).
- **No Duplicates**: TreeSet does not allow duplicate elements.
- **No Null Values**: It does **not allow null** values if even one comparator is used.
- **Logarithmic Time Complexity**: Operations like add(), remove(), and contains() take **O(log n)** time.
- **NavigableSet Features**: Supports methods like ceiling(), floor(), higher(), and lower().

```java
import java.util.TreeSet;

public class TreeSetExample {
    public static void main(String[] args) {
        TreeSet<Integer> treeSet = new TreeSet<>();

        // Adding elements
        treeSet.add(20);
        treeSet.add(10);
        treeSet.add(30);
        treeSet.add(10); // Duplicate, will be ignored

        System.out.println("TreeSet: " + treeSet);
        // Output: [10, 20, 30] - Sorted and unique
    }
}
```

Method	Description
`add(E e)`	Adds an element to the set.
`remove(Object o)`	Removes a specific element from the set.
`contains(Object o)`	Returns `true` if the set contains the specified element.
`first()`	Returns the first (lowest) element.
`last()`	Returns the last (highest) element.
`higher(E e)`	Returns the lowest element greater than `e`.
`lower(E e)`	Returns the greatest element less than `e`.
`ceiling(E e)`	Returns the lowest element greater than or equal to `e`.
`floor(E e)`	Returns the greatest element less than or equal to `e`.
`pollFirst()`	Retrieves and removes the first element.
`pollLast()`	Retrieves and removes the last element.

```java
import java.util.TreeSet;

public class TreeSetExample {
    public static void main(String[] args) {
        TreeSet<Integer> treeSet = new TreeSet<>();
        treeSet.add(10);
        treeSet.add(20);
        treeSet.add(30);
        treeSet.add(40);
        treeSet.add(50);

        System.out.println("TreeSet: " + treeSet);

        // higher() - Returns the smallest element greater than the given element
        System.out.println("Higher than 30: " + treeSet.higher(30)); // Output: 40

        // lower() - Returns the largest element smaller than the given element
        System.out.println("Lower than 30: " + treeSet.lower(30)); // Output: 20

        // ceiling() - Returns the smallest element greater than or equal to the given element
        System.out.println("Ceiling of 35: " + treeSet.ceiling(35)); // Output: 40

        // floor() - Returns the largest element smaller than or equal to the given element
        System.out.println("Floor of 35: " + treeSet.floor(35)); // Output: 30

        // pollFirst() - Removes and returns the first (lowest) element
        System.out.println("Poll First: " + treeSet.pollFirst()); // Output: 10
        System.out.println("After pollFirst: " + treeSet);

        // pollLast() - Removes and returns the last (highest) element
        System.out.println("Poll Last: " + treeSet.pollLast()); // Output: 50
        System.out.println("After pollLast: " + treeSet);
    }
}
```

snappify.com

List in Java

✅ Overview

In Java, a **List** is an ordered collection (also known as a **sequence**) that allows you to store **duplicate** elements. It is a part of the **Java Collections Framework** and is defined in the java.util package.

The List interface extends the Collection interface and provides methods to manipulate elements based on their **index**.

📌 Key Characteristics of List

- **Ordered Collection**: Maintains the **insertion order** of elements.
- **Allows Duplicates**: You can store multiple instances of the same element.
- **Indexed Access**: Provides **index-based** access and manipulation using methods like get(), set(), add(), and remove().
- **Null Values**: Allows storing null values.
- **Resizable**: Some implementations like ArrayList and LinkedList are dynamically resizable.

1.LinkedList in Java

✅ Overview

A **LinkedList** in Java is a part of the java.util package and implements the **List** and **Deque** interfaces. It is a type of **doubly linked list** that stores elements in nodes, where each node contains:

- **Data (Element)**: The actual value.
- **Pointer to Next Node**: Reference to the next node in the list.
- **Pointer to Previous Node**: Reference to the previous node (because it's a **doubly linked list**).

📌 Key Characteristics of LinkedList

- **Ordered Collection**: Maintains the insertion order of elements.
- **Allows Duplicates**: Can store duplicate elements.
- **Efficient for Insertions and Deletions**: Adding or removing elements from the beginning or middle is faster compared to ArrayList.
- **Not Efficient for Search Operations**: Accessing an element by index takes **O(n)** time.
- **Implements Queue and Deque**: Supports operations like offer(), poll(), peek() for queue-like behavior.

Method	Description
`add(E e)`	Adds an element to the end of the list.
`addFirst(E e)`	Adds an element to the beginning of the list.
`addLast(E e)`	Adds an element to the end of the list.
`get(int index)`	Retrieves the element at the specified index.
`remove(int index)`	Removes the element at the specified index.
`removeFirst()`	Removes the first element from the list.
`removeLast()`	Removes the last element from the list.
`contains(Object o)`	Checks if the list contains the specified element.
`size()`	Returns the number of elements in the list.
`isEmpty()`	Checks if the list is empty.
`clear()`	Removes all elements from the list.
`peek()`	Retrieves the first element without removing it.
`poll()`	Retrieves and removes the first element.



```java
import java.util.LinkedList;

public class LinkedListDemo {
    public static void main(String[] args) {
        LinkedList<String> list = new LinkedList<>();

        // Adding elements
        list.add("Java");
        list.add("Python");
        list.add("C++");

        System.out.println("Initial List: " + list);

        // Adding elements at specific positions
        list.addFirst("HTML");
        list.addLast("JavaScript");

        System.out.println("After addFirst and addLast: " + list);

        // Accessing elements
        System.out.println("First Element: " + list.getFirst());
        System.out.println("Last Element: " + list.getLast());

        // Removing elements
        list.remove("Python"); // Removes by value
        System.out.println("After Removing 'Python': " + list);

        list.removeFirst();
        System.out.println("After removeFirst(): " + list);

        list.removeLast();
        System.out.println("After removeLast(): " + list);

        // Checking size
        System.out.println("List Size: " + list.size());

        // Checking if list contains a specific element
        System.out.println("Contains 'Java': " + list.contains("Java"));
    }
}
```


2.Array List(give above)

3.vector(give above)

Map in Java

✅ Overview

A **Map** in Java is a part of the java.util package and is used to store key-value pairs. It allows you to efficiently retrieve, update, or remove values based on their **keys**.

- **Keys are unique**: A Map cannot contain duplicate keys.
- **Values can be duplicated**: Multiple keys can have the same value.
- **No Insertion Order Guarantee**: Except for certain implementations like LinkedHashMap.

1.HashMap in Java

✅ Overview

A **HashMap** in Java is a part of the java.util package and implements the **Map** interface. It is used to store data in **key-value** pairs, where each key is unique, and it provides constant-time performance for basic operations like **add**, **remove**, and **lookup**.

- **Key Features**:
    - **No Order Guarantee**: HashMap does not maintain the order of elements.
    - **Allows Null Values and One Null Key**: You can have one null key and multiple null values.
    - **Efficient**: Operations like get, put, and remove generally take **O(1)** time.
    - **Non-Synchronized**: Not thread-safe. Use Collections.synchronizedMap() or ConcurrentHashMap for thread safe

Method	Description
`put(K key, V value)`	Adds or updates the key-value pair.
`get(Object key)`	Returns the value for the specified key.
`remove(Object key)`	Removes the key-value pair for the specified key.
`containsKey(Object key)`	Checks if the map contains the specified key.
`containsValue(Object value)`	Checks if the map contains the specified value.
`size()`	Returns the number of key-value pairs.
`isEmpty()`	Checks if the map is empty.
`clear()`	Removes all key-value pairs from the map.
`keySet()`	Returns a set of all the keys.
`values()`	Returns a collection of all the values.
`entrySet()`	Returns a set of key-value pairs (Map.Entry).



2. TreeMap in Java

✅ Overview

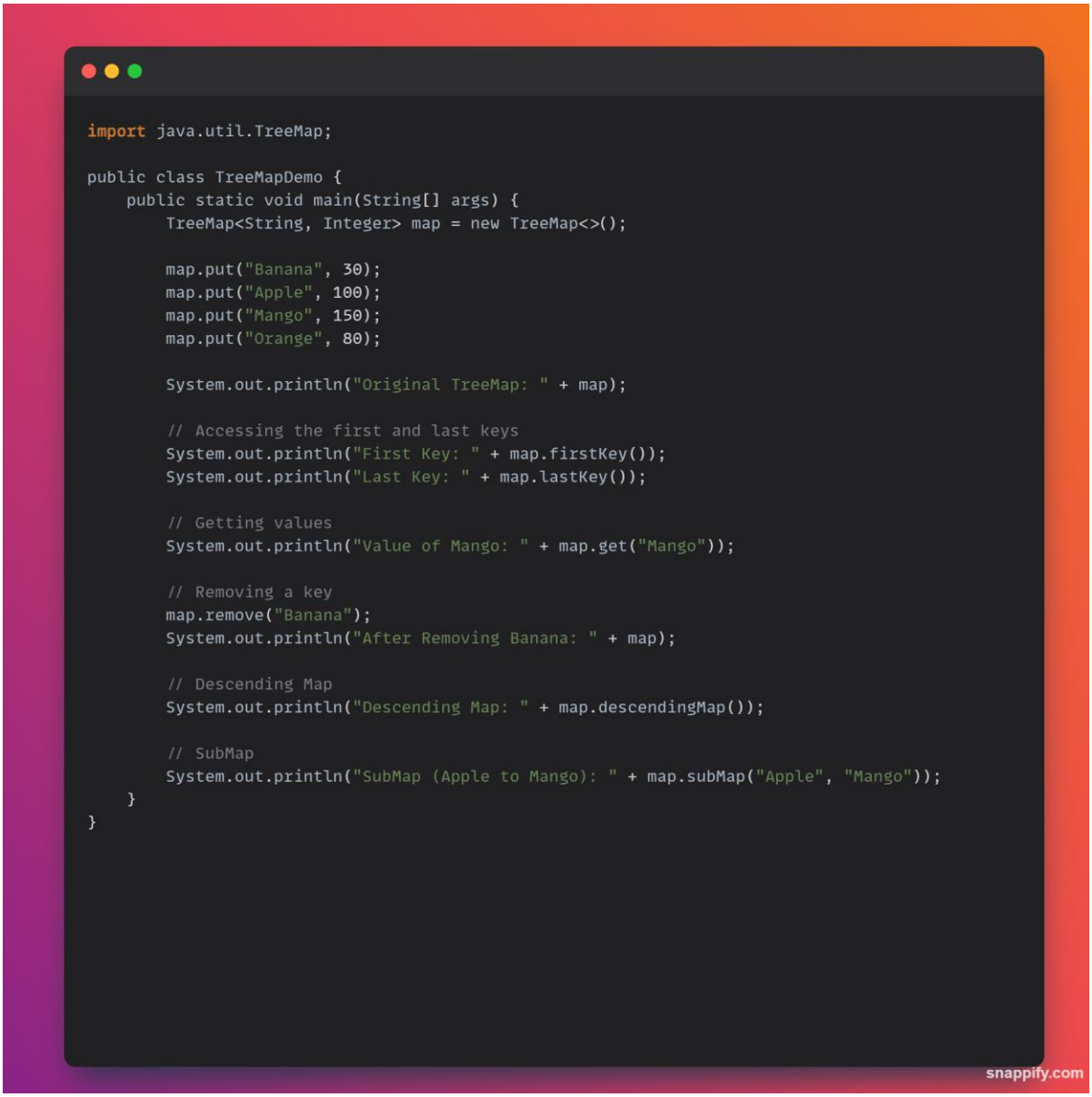
A **TreeMap** in Java is a part of the java.util package and implements the **NavigableMap** interface, which extends the **SortedMap** interface. It stores key-value pairs in a **sorted order** using a **Red-Black Tree** (a self-balancing binary search tree).

- **Sorted Order**: Keys are maintained in natural order (ascending) or using a custom comparator.
- **No Null Keys Allowed**: Unlike HashMap, TreeMap does not allow null keys.
- **Null Values Allowed**: You can store null values.

- **Efficient**: Provides **O(log n)** time complexity for get(), put(), remove(), and other operations.



Method	Description
`put(K key, V value)`	Inserts a key-value pair. If the key exists, it updates the value.
`get(Object key)`	Retrieves the value associated with the key.
`remove(Object key)`	Removes the key-value pair for the specified key.
`containsKey(Object key)`	Checks if the map contains the specified key.
`containsValue(Object value)`	Checks if the map contains the specified value.
`size()`	Returns the number of key-value pairs.
`isEmpty()`	Checks if the map is empty.
`clear()`	Removes all key-value pairs.
`firstKey()`	Returns the first (smallest) key.
`lastKey()`	Returns the last (largest) key.
`pollFirstEntry()`	Removes and returns the first key-value pair.
`pollLastEntry()`	Removes and returns the last key-value pair.
`subMap(K fromKey, K toKey)`	Returns a view of the portion of the map from `fromKey` to `toKey`.
`descendingMap()`	Returns a reverse 'ered view of the map.



📝 Annotations in Java

✅ Overview

Annotations in Java are special **metadata** that provide additional information about the code. They don't directly affect program logic but are used by the compiler, development tools, or runtime environments for specific tasks.

- **Introduced in**: Java 5 (@Override, @Deprecated, @SuppressWarnings)
- **Enhanced in**: Java 8 with support for type annotations (@Target and @Retention)

📦 Why Use Annotations?

- Provide **instructions to the compiler** (e.g., suppress warnings, indicate overrides).
- Help with **code documentation**.

🛠️ Built-in Annotations in Java

Java provides several useful annotations:

✅ 1. @Override

- Used to ensure a method is overriding a superclass method.
- Helps prevent errors when method names or signatures are incorrect

```
class Parent {
    void display() {
        System.out.println("Display from Parent");
    }
}

class Child extends Parent {
    @Override
    void display() {
        System.out.println("Display from Child");
    }
}
```

✅ 2. @Deprecated

- Marks a method, class, or field as obsolete.
- Compiler generates a warning when it is used.
    - to indicate that the code should no longer be used because it might be removed or replaced in the future. While it may still work, it's recommended to use an alternative.

```java
class OldClass {
    @Deprecated
    public void oldMethod() {
        System.out.println("This method is deprecated.");
    }
}

public class Test {
    public static void main(String[] args) {
        OldClass obj = new OldClass();
        obj.oldMethod(); // Warning: Deprecated method
    }
}
```

3. @SuppressWarnings

- Suppresses compiler warnings.

```java
import java.util.*;

public class Example {
    @SuppressWarnings("unchecked")
    public void show() {
        List rawList = new ArrayList();
        rawList.add("Hello");
        rawList.add(100); // No warning shown
        System.out.println(rawList);
    }
}
```

4. @FunctionalInterface

- Used to define a **Functional Interface** (an interface with one abstract method).
- Ensures no additional abstract methods are added accidentally.

```
@FunctionalInterface
interface MyFunction {
    void execute();
}
```

You can also create your own annotations using @interface.

```
@interface MyAnnotation {
    String value() default "Default Message";
}
```

Unit 4:

🏗️ Java Foundation Classes (JFC)

Java Foundation Classes (JFC) are a set of libraries in **Java** used for building **graphical user interfaces (GUIs)** and adding rich features to desktop applications.

JFC is part of the **Java Standard Edition (JSE)** and consists of multiple APIs that provide components and tools for developing cross-platform GUI applications.

🗂️ Components of JFC

JFC consists of the following main components:

1. **AWT (Abstract Window Toolkit)**
2. **Swing**
3. **Java 2D API**
4. **Accessibility API**

AWT (Abstract Window Toolkit)

AWT (Abstract Window Toolkit) in Java, which is a part of the Java Foundation Classes (JFC) used for creating **Graphical User Interfaces (GUIs)**.

Here's a simplified summary:

✅ What is AWT?

- **AWT** is a platform-dependent, GUI toolkit in Java.
- It provides basic components like **buttons**, **labels**, **text fields**, and **checkboxes** to build desktop applications.
- Components rely on the **native system GUI** (called **peers**), leading to platform-specific appearance.

🗂️ Key Features of AWT

1. **Platform-Dependent**: Uses native UI components from the operating system.
2. **Event Handling**: Supports event-driven programming using **Event Listeners**.

3. **Layout Managers**: Controls how components are arranged within containers.
4. **Heavyweight Components**: Because they rely on native GUI components, they consume more resources.

🛠 Basic Components in AWT

- **Button**: Button b = new Button("Click Me");
- **Label**: Label label = new Label("Hello, AWT!");
- **TextField**: TextField tf = new TextField(20);
- **Checkbox**: Checkbox check = new Checkbox("Accept Terms");
- **Frame**: Frame f = new Frame("AWT Frame Example");

```
import java.awt.*;
import java.awt.event.*;

public class AWTExample extends Frame {
    public AWTExample() {
        Button b = new Button("Click Me");

        b.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                System.out.println("Button Clicked!");
            }
        });

        add(b);
    }

    public static void main(String[] args) {
        new AWTExample();
    }
}
```

📐 Layout Managers in AWT

AWT provides Layout Managers to organize components in a container:

1. **FlowLayout**: Arranges components in a row.
2. **BorderLayout**: Divides the container into five regions (**North**, **South**, **East**, **West**, **Center**).
3. **GridLayout**: Arranges components in a grid of rows and columns.

4. **CardLayout**: Displays one component at a time like a card stack.

Swing in Java

- **Swing** is a part of the **Java Foundation Classes (JFC)** used for building **Graphical User Interfaces (GUIs)** in Java applications.
- It provides a set of lightweight, platform-independent components.
- Swing is built on top of **AWT (Abstract Window Toolkit)** but offers more powerful and flexible UI components.
- All Swing components are located in the javax.swing package.

Key Features of Swing

- **Lightweight:** Unlike AWT, Swing components do not depend on native platform-specific code.
- **Pluggable Look and Feel:** You can customize the appearance using different themes.
- **Event-Driven Programming:** Supports event handling with listeners.
- **Rich Set of Components:** Provides UI components like labels, buttons, text fields, tables, and trees.
- **MVC Architecture:** Swing follows the **Model-View-Controller** pattern for managing components.

```java
import javax.swing.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class SimpleSwingApp {
    public static void main(String[] args) {
        // Create a JFrame (Window)
        JFrame frame = new JFrame("Swing Example");


        // Create a JButton
        JButton button = new JButton("Click Me!");
        button.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                JOptionPane.showMessageDialog(frame, "Button Clicked!");
            }
        });

        // Add button to frame
        frame.add(button);

    }
}
```

Layout Manager	Description
`FlowLayout`	Places components in a row, wrapping as necessary.
`BorderLayout`	Divides container into five regions: **North, South, East, West, Center.**
`GridLayout`	Arranges components in a grid with equal-sized cells.
`BoxLayout`	Aligns components either vertically or horizontally.
`CardLayout`	Allows switching between multiple panels like a card deck.

3. Java 2D API

- Java 2D API is used for **2D graphics and imaging**.
- Provides advanced capabilities like drawing shapes, text rendering, and image processing.
- Located in the java.awt and java.awt.geom packages.

```java
Graphics2D g2 = (Graphics2D) g;
g2.setColor(Color.BLUE);
g2.fillRect(50, 50, 200, 100);
```

4. Accessibility API

- Provides support for **assistive technologies** like screen readers or voice recognition systems.
- Allows developers to create applications accessible to users with disabilities.
- Part of javax.accessibility package.

Feature	Swing	AWT (Abstract Window Toolkit)
Component Type	Lightweight (Drawn by Java)	Heavyweight (Uses native OS components)
Platform Dependency	Platform-independent	Platform-dependent
Performance	Slower due to rich features	Faster for basic applications
Look and Feel	Customizable using Pluggable Look & Feel	Native OS Look and Feel
Package	`javax.swing`	`java.awt`
Event Handling	Improved and simplified using Listeners	Basic event handling using Listeners
Advanced Components	Supports complex components like JTable, JTree	Provides basic components like Button, Label
Memory Usage	Higher (more resources used)	Lower (less memory consumption)
Extensibility	Highly extensible and customizable	Limited customization
Thread Safety	Not thread-safe (Use SwingUtilities)	Not thread-safe
Container	Provides lightweight containers like JPanel	Uses heavy containers like Frame, Panel
Graphics Support	Supports 2D Graphics using Java 2D API	Limited Graphics Support
Best For	Complex and modern GUI applications	Simple GUI applications

Basic Swing Containers in Java

In **Swing**, containers are used to hold and organize other components like buttons, labels, and text fields. There are **three primary Swing containers** that serve as the foundation for GUI applications.

✅ 1. JFrame

- **JFrame** is a top-level container that represents a window with a title bar, minimize, maximize, and close buttons.
- It is typically used as the main window for desktop applications.

```java
import javax.swing.*;

public class JFrameExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("My First JFrame");
        frame.setSize(400, 300);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Explanation:

- JFrame("Title") → Creates a window with a title.
- setSize() → Sets the size of the window.
- setDefaultCloseOperation() → Closes the application when the window is closed.
- setVisible(true) → Displays the window.

✅ 2. JPanel

- **JPanel** is a generic container used to group components together.
- It doesn't have a title bar or window border.
- Often used inside JFrame to organize and layout components.

```java
import javax.swing.*;
import java.awt.*;

public class JPanelExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("JPanel Example");
        frame.setSize(400, 300);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // Create a JPanel with FlowLayout
        JPanel panel = new JPanel();
        panel.add(new JButton("Button 1"));
        panel.add(new JButton("Button 2"));
        panel.add(new JButton("Button 3"));

        frame.add(panel);
        frame.setVisible(true);
    }
}
```

✅ 3. JDialog

- **JDialog** is a pop-up window often used for messages, confirmations, or custom dialogs.
- It can be **modal** (blocks user interaction with other windows until closed) or **modeless**.

```java
import javax.swing.*;

public class JDialogExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("JDialog Example");
        frame.setSize(400, 300);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);

        // Create JDialog
        JDialog dialog = new JDialog(frame, "Custom Dialog", true);
        dialog.setSize(200, 150);
        dialog.add(new JLabel("This is a dialog box"));
        dialog.setVisible(true);
    }
}
```

Explanation:

- JDialog(frame, "Title", true) → Creates a modal dialog (blocks the parent window).
- add() → Adds a label to the dialog.
- setVisible(true) → Displays the dialog.

Swing Components in Java

✅ Overview

Swing is a part of the **Java Foundation Classes (JFC)** and provides a rich set of **GUI components** to build desktop applications. All Swing components are lightweight and platform-independent.

- All Swing components are located in the **javax.swing** package.
- They provide a consistent look and feel across platforms.
- Swing follows the **MVC (Model-View-Controller)** architecture, making it flexible and customizable.

Component	Description
`JFrame`	Creates the main application window.
`JPanel`	A container to hold and organize components.
`JButton`	A clickable button.
`JLabel`	Displays text or images.
`JTextField`	Single-line input field for text.
`JTextArea`	Multi-line input field for text.
`JCheckBox`	A selectable check box for boolean input.
`JRadioButton`	A button used in a group where only one can be selected.
`JComboBox`	A drop-down list of selectable options.
`JList`	Displays a list of items for selection.
`JTable`	Displays tabular data.
`JScrollPane`	Adds scrollbars to components.
`JMenuBar`	Adds a menu bar to the application.
`JProgressBar`	Displays progress information.
`JSlider`	Allows users to select a value within a range.
`JDialog`	A pop-up dialog window.

```java
// JLabel
        JLabel label = new JLabel("Enter your name:");
        add(label);

        // JTextField
        JTextField textField = new JTextField(20);
        add(textField);

        // JButton
        JButton button = new JButton("Submit");
        button.addActionListener(e -> JOptionPane.showMessageDialog(this, "Hello, " + textField.ge
        add(button);

        // JCheckBox
        JCheckBox checkBox = new JCheckBox("Accept Terms and Conditions");
        add(checkBox);

        // JRadioButton
        JRadioButton male = new JRadioButton("Male");
        JRadioButton female = new JRadioButton("Female");
        ButtonGroup group = new ButtonGroup();
        group.add(male);
        group.add(female);
        add(male);
        add(female);

        // JComboBox
        String[] countries = {"India", "USA", "UK", "Canada"};
        JComboBox<String> comboBox = new JComboBox<>(countries);
        add(comboBox);

        // JList
        String[] languages = {"Java", "Python", "C++", "JavaScript"};
        JList<String> list = new JList<>(languages);
        add(new JScrollPane(list));

        // JTable
        String[][] data = {{"John", "25"}, {"Sara", "22"}};
        String[] columns = {"Name", "Age"};
        JTable table = new JTable(data, columns);
        add(new JScrollPane(table));

        setVisible(true);
```

snappify.com

Event Handling in Java

Event Handling in Java is a mechanism that responds to user actions, such as:

- **Button clicks**
- **Mouse movements**
- **Keyboard inputs**
- **Window events**

Java uses the **Event Delegation Model** to handle events, meaning:

- **Event Source** → The object that generates an event (e.g., Button, Checkbox).
- **Event Object** → Contains information about the event (e.g., ActionEvent, MouseEvent).
- **Event Listener** → An interface that defines methods to respond to the event.

Component	Description
`EventObject`	The root class for all event objects.
`ActionEvent`	Generated when a button is clicked.
`MouseEvent`	Generated when a mouse is clicked, moved, or dragged.
`KeyEvent`	Generated when a key is pressed, released, or typed.
`WindowEvent`	Generated when a window is opened, closed, or resized.
`ActionListener`	Listens for button clicks or menu selections.
`MouseListener`	Handles mouse click, press, or release events.
`KeyListener`	Responds to keyboard events.
`WindowListener`	Monitors window actions like closing or minimizing.

ActionListener

```
button = new Button("Click Me");

// Register the button with ActionListener
button.addActionListener(this);

// Add the button to the frame
add(button);
```

```
// Perform action when button is clicked
public void actionPerformed(ActionEvent e) {
    System.out.println("Button Clicked!");
}
```

MouseListener

```
addMouseListener(this);

// Implementing MouseListener methods
public void mouseClicked(MouseEvent e) {
    label.setText("Mouse Clicked");
}

public void mouseEntered(MouseEvent e) {
    label.setText("Mouse Entered the Frame");
}

public void mouseExited(MouseEvent e) {
    label.setText("Mouse Exited the Frame");
}
```

KeyListener

```
addKeyListener(this);
 public void keyPressed(KeyEvent e) {
       label.setText("Key Pressed: " + e.getKeyChar());
   }

   public void keyReleased(KeyEvent e) {
       label.setText("Key Released: " + e.getKeyChar());
   }
```

Applet in java

✅ Overview

An **applet** is a Java program that runs inside a web browser or an applet viewer.
Although **Applets** were commonly used in older web applications, they are now rarely used and are considered deprecated in modern Java versions.
However, understanding applets is still useful for legacy systems and learning how GUI components work.

- Swing provides a class called **JApplet** in the javax.swing package, which allows developers to create applets using Swing components.
- Unlike AWT applets, JApplet supports all the advantages of Swing, such as **lightweight components** and **pluggable look-and-feel**.

📦 Key Features of Swing Applets

- Swing applets use JApplet instead of Applet.
- They are more flexible and customizable compared to AWT-based applets.
- Can include components like JButton, JLabel, JTextField, etc.
- Provide better graphics and event handling using Swing.

Applet Methods:(Life cycle)

■init( ) - initialization

■ start( ) - resume processing (e.g. animations)

■ paint( ) - redraw stuff (‘expose’ event)

■ stop( ) - pause

■ destroy( ) - cleanup

🔄 Applet Life Cycle Flow

Here’s how the life cycle methods are called:

1. **Initialization (init())**
    - Executed once when the applet is loaded.
2. **Starting (start())**
    - Executed when the applet becomes visible.
3. **Painting (paint())**
    - Executed whenever the applet needs to be redrawn, like during resizing.
4. **Stopping (stop())**
    - Executed when the applet is minimized or hidden.
5. **Destruction (destroy())**
    - Executed when the applet is closed to release resources.



```java
import java.applet.*;
import java.awt.*;

/*
<applet code="LifecycleApplet.class" width="400" height="300"></applet>
*/

public class LifecycleApplet extends Applet {

    // Called once during applet initialization
    public void init() {
        System.out.println("Applet Initialized (init)");
    }

    // Called when the applet starts or becomes visible
    public void start() {
        System.out.println("Applet Started (start)");
    }

    // Called to repaint the applet on the screen
    public void paint(Graphics g) {
        System.out.println("Applet Painted (paint)");
        g.drawString("Hello, Applet Life Cycle!", 50, 150);
    }

    // Called when the applet is stopped or moved to the background
    public void stop() {
        System.out.println("Applet Stopped (stop)");
    }

    // Called when the applet is destroyed
    public void destroy() {
        System.out.println("Applet Destroyed (destroy)");
    }
}
```


What are the Restrictions imposed on Java Applets?

1.file restriction

2.network restriction

3.window restriction

4.printer and clipboard restriction

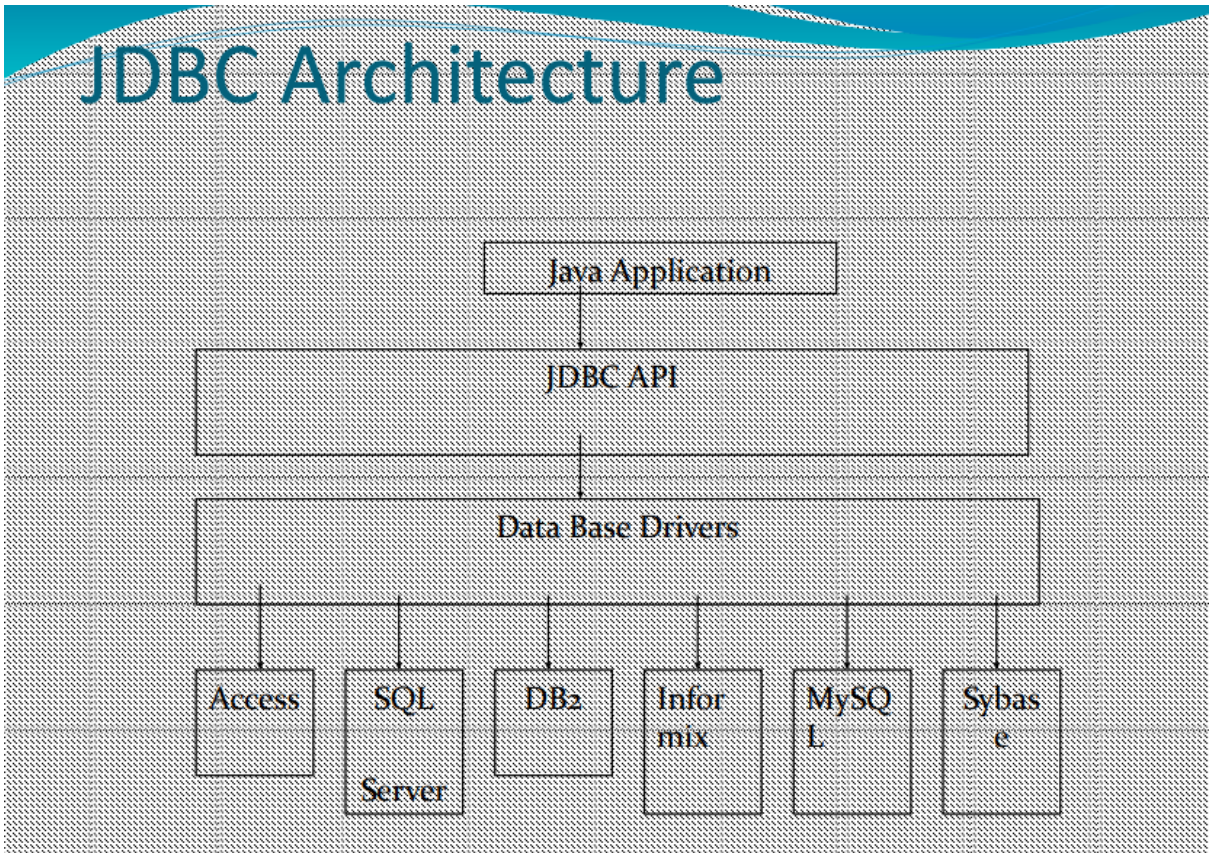
5.sign in -signup restriction

6.security access restriction

Unit 5:

Introduction to JDBC

- JDBC is a Sun trademark It is often taken to stand for Java Database Connectivity
- Java is very standardized, but there are many versions of SQL
- JDBC is a means of accessing SQL databases from Java
- JDBC is a standardized API for use by Java programs
- JDBC is also a specification for how third-party vendors should write database drivers to access specific SQL versions



Java Application

- This is the client-side application written in Java that uses JDBC to interact with a database.
- It can be a desktop app, web app, or enterprise app.

JDBC API

- A collection of Java interfaces and classes (java.sql.*) that provide database connectivity.
- Methods like Connection, Statement, PreparedStatement, and ResultSet are part of this API.

 JDBC Driver Manager

- Responsible for managing and selecting the appropriate database driver for the application.
- Connects the application to the database using DriverManager.getConnection().

 Database Driver

- A driver is a software component that translates JDBC calls to database-specific commands.
- Different databases require different drivers (e.g., MySQL Connector/J for MySQL).

 Database

- The actual relational database (like MySQL, Oracle, or PostgreSQL) that stores data using SQL.

Types of Driver :

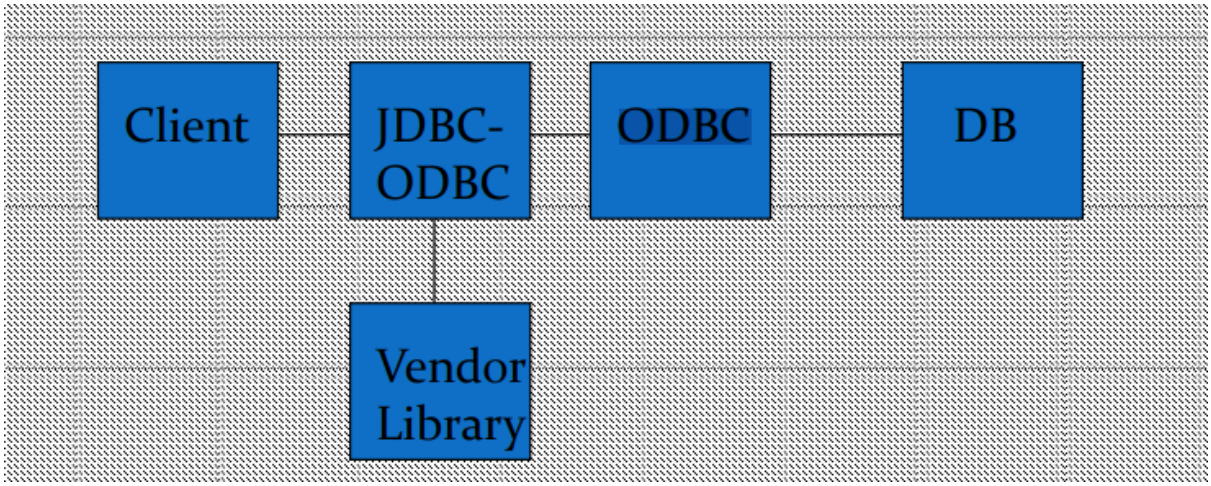
✅ **1. Type 1: JDBC-ODBC Bridge Driver**

📌 **Description**

- The **Type 1 Driver** uses the **ODBC (Open Database Connectivity)** interface to communicate with the database.
- It acts as a bridge between **JDBC** and **ODBC**.
- It uses the **ODBC driver** installed on the machine to connect to the database.

🛠 **How It Works**

1. JDBC API sends the query to the Type 1 driver.
2. Type 1 driver translates it into an ODBC call.
3. ODBC driver then interacts with the database and sends results back to JDBC.




```
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
Connection con = DriverManager.getConnection("jdbc:odbc:myDatabase", "username", "password");
```

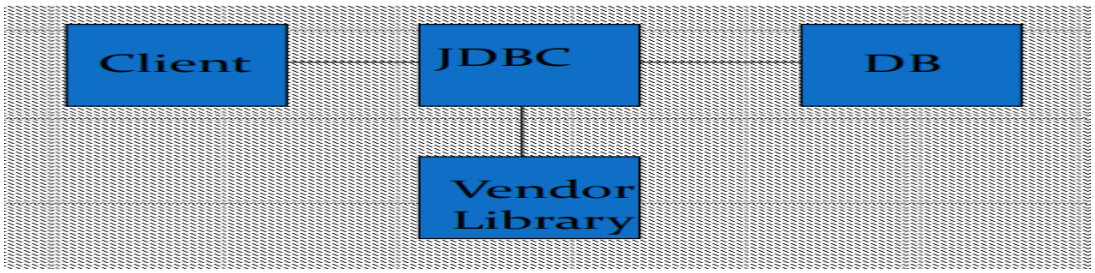
✅ 2. Type 2: Native API Driver

📌 Description

- The **Type 2 Driver** uses **native API** (database-specific client libraries) to communicate directly with the database.
- It requires the database's native client library installed on the client machine.

🛠 How It Works

1. JDBC API sends the query to the Type 2 driver.
2. The driver uses native API calls to interact with the database.
3. The results are sent back to JDBC.



```
Class.forName("oracle.jdbc.driver.OracleDriver");
Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe", "username
```

✅ 3. Type 3: Network Protocol Driver (Middleware Driver)

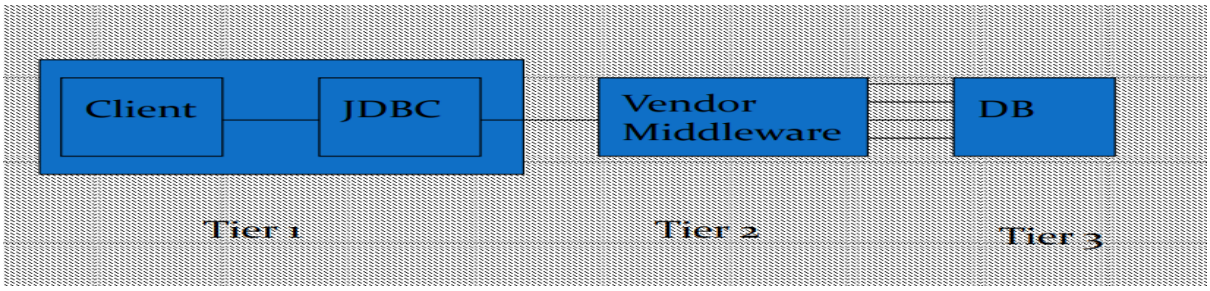
📌 Description

- The **Type 3 Driver** uses a **network protocol** to communicate with a middleware server.
- The middleware server converts JDBC calls to the database-specific protocol.

🛠 How It Works

1. JDBC API sends the query to the Type 3 driver.
2. The driver forwards the request to a **middleware server** using a network protocol.
3. The middleware translates the query to the database and sends results back.

```
Class.forName("com.ids.sql.IDSDriver");
Connection con = DriverManager.getConnection("jdbc:ids://localhost:12/conn?dsn='mydb'");
```



✅ 4. Type 4: Thin Driver (Pure Java Driver)

📌 Description

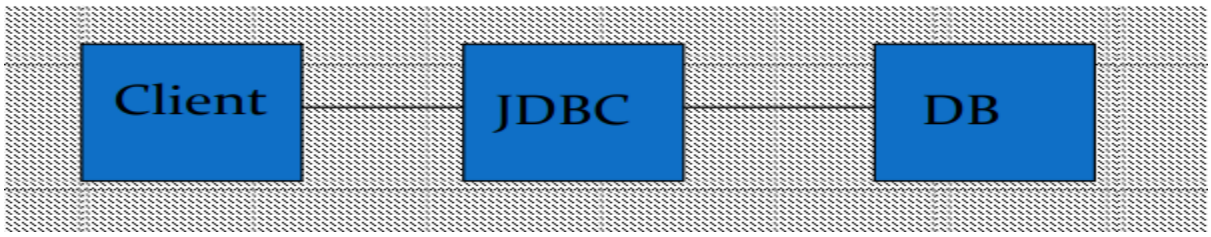
- The **Type 4 Driver** is a **pure Java driver** that directly communicates with the database using database-specific protocols.
- It converts JDBC calls into database-specific calls without requiring any middleware or native code.

🛠 How It Works

1. JDBC API sends the query to the Type 4 driver.
2. The driver directly interacts with the database using its native network protocol.

3. The results are sent back to JDBC.

```java
Class.forName("com.mysql.cj.jdbc.Driver");
Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/myDatabase", "usern
```



Loading and register driver:

Class.forName("com.mysql.cj.jdbc.Driver");

Connection con =DriverManager.getConnection("jdbc:mysql://localhost:3306/myDatabase", "username", "password");

Connection in Java:

Step 1: Import the JDBC Packages

- You need to import the necessary classes from the java.sql package.

✅ **Step 2: Load and Register the Driver**

- JDBC uses **drivers** to connect to the database.
- Use the Class.forName() method to load the driver into memory.

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

Step 3: Establish the Connection

- Use DriverManager.getConnection() to establish a connection.
- Provide the **database URL**, **username**, and **password**

```
String url = "jdbc:mysql://localhost:3306/mydatabase";
String user = "root";
String password = "1234";

Connection con = DriverManager.getConnection(url, user, password);
System.out.println("Connected to Database");
```

Step 4: Create and Execute a Statement

- After establishing the connection, create a **Statement** or **PreparedStatement** to execute SQL queries.
- Use createStatement() for basic SQL or prepareStatement() for parameterized SQL.

```
Statement stmt = con.createStatement();
String query = "SELECT * FROM employees";
ResultSet rs = stmt.executeQuery(query);
```

Step 6: Close the Resources

Con.close();

Statement In java

📝 **Statement in Java JDBC**

✅ **Overview**

In **JDBC (Java Database Connectivity)**, the Statement interface is used to execute **SQL queries** against a database. It is a part of the java.sql package and is typically used for executing **static SQL statements** like SELECT, INSERT, UPDATE, and DELETE.

- It is created using a Connection object.
- The results of the SQL query are stored in a ResultSet object for **SELECT** queries.

Types of Statements in JDBC

JDBC provides three types of statements:

Statement Type	Description	When to Use
Statement	Used for executing static SQL queries without parameters.	When queries are fixed and do not change.
PreparedStatement	Used for executing parameterized SQL queries.	When you need to execute the same query multiple times with different values.
CallableStatement	Used for executing stored procedures from the database.	When working with stored procedures or functions.

1.exmple of create statement:

```
public void display() {
       try {
           Connection con = getConnection();
           Statement st = con.createStatement();
           ResultSet rs = st.executeQuery("SELECT * FROM testtb");

           System.out.println("\nID\tName\tAge");
           System.out.println("-----------------------");

           while (rs.next()) {
               System.out.println(rs.getInt("id") + "\t" + rs.getString("name") + "\t" + rs.getInt("age"));
           }

       } catch (SQLException ex) {
           Logger.getLogger(JavaApplication1.class.getName()).log(Level.SEVERE, null, ex);
       }
   }
```

2.exmple of PreparedStatement

```
public void insert() {
        try {
            Connection con = getConnection();

            System.out.print("\nEnter Name: ");
            String name = sc.next();

            System.out.print("Enter Age: ");
            int age = sc.nextInt();

            String insert = "INSERT INTO testtb(name, age) VALUES(?, ?)";
            PreparedStatement pst = con.prepareStatement(insert);
            pst.setString(1, name);
            pst.setInt(2, age);

            int row = pst.executeUpdate();
            if (row > 0) {
                System.out.println("Record inserted successfully.");
            }

        } catch (SQLException ex) {
            System.out.println("Error: " + ex.getMessage());
        }
    }
```

3.exmple of callablestatement

```
public void getAuthorByISSN() {
        try {
            Connection con = getConnection();

            System.out.print("\nEnter Book ISSN Number: ");
            String issnNo = sc.next();

            CallableStatement stmt = con.prepareCall("{CALL GetAuthorByISSN(?)}");
            stmt.setString(1, issnNo);

            ResultSet rs = stmt.executeQuery();

            if (rs.next()) {
                System.out.println("\nAuthor Name: " + rs.getString("author_name"));
            } else {
                System.out.println("\nNo author found for this ISSN.");
            }

            // Close resources
            rs.close();
            stmt.close();
            con.close();
        } catch (SQLException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
```

✅ Executing SQL Queries with Statement

The `Statement` interface provides three primary methods to execute SQL queries:

Method	Description	Use Case
`executeQuery(String sql)`	Executes SQL `SELECT` queries and returns a `ResultSet`.	When fetching data from the database.
`executeUpdate(String sql)`	Executes SQL `INSERT`, `UPDATE`, or `DELETE` queries.	When modifying data.
`execute(String sql)`	Can execute any type of SQL query.	When using unknown or dynamic SQL queries.

🔎 Transaction Methods in JDBC

☑ Overview

A **transaction** in JDBC refers to a sequence of one or more SQL operations executed as a **single unit of work**.

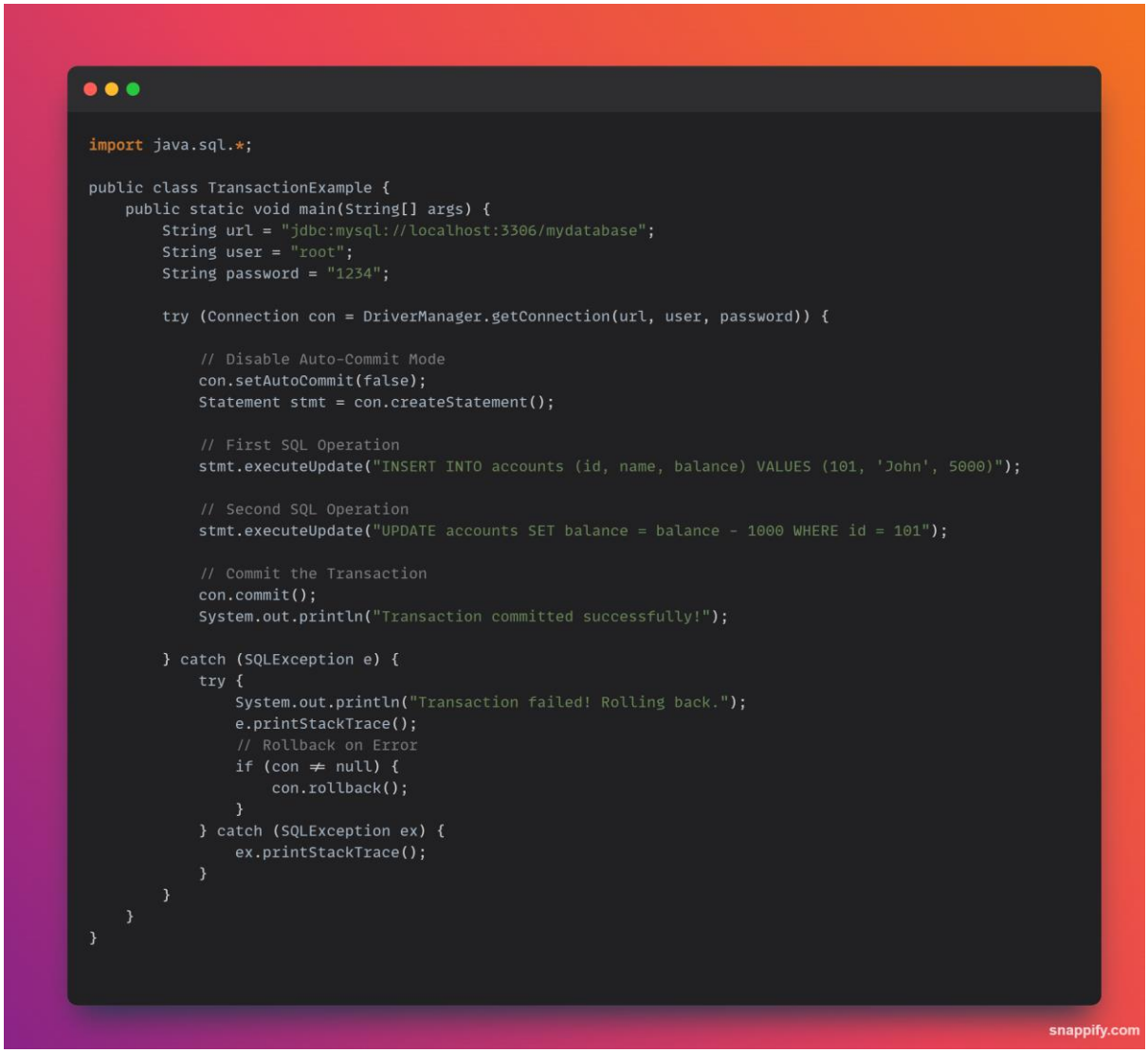
- It ensures **data integrity** and follows the principles of **ACID** (**Atomicity, Consistency, Isolation, Durability**).
- In JDBC, transactions are managed using the Connection interface.

By default, JDBC uses **Auto-Commit Mode**, meaning each SQL statement is treated as a transaction and is automatically committed to the database.

🗃️ Transaction Control Methods in JDBC

The Connection interface provides several methods to manage transactions:

Method	Description
`setAutoCommit(boolean)`	Enables or disables auto-commit mode. `false` means manual commit.
`commit()`	Commits the current transaction and makes changes permanent.
`rollback()`	Rolls back (undoes) changes since the last commit.
`setSavepoint(String)`	Creates a **Savepoint** to mark a point in the transaction.
`rollback(Savepoint)`	Rolls back to a specific savepoint, undoing all changes after it.
`releaseSavepoint(Savepoint)`	Removes a previously created savepoint.



```java
import java.sql.*;

public class TransactionExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/mydatabase";
        String user = "root";
        String password = "1234";

        try (Connection con = DriverManager.getConnection(url, user, password)) {

            // Disable Auto-Commit Mode
            con.setAutoCommit(false);
            Statement stmt = con.createStatement();

            // First SQL Operation
            stmt.executeUpdate("INSERT INTO accounts (id, name, balance) VALUES (101, 'John', 5000)");

            // Second SQL Operation
            stmt.executeUpdate("UPDATE accounts SET balance = balance - 1000 WHERE id = 101");

            // Commit the Transaction
            con.commit();
            System.out.println("Transaction committed successfully!");

        } catch (SQLException e) {
            try {
                System.out.println("Transaction failed! Rolling back.");
                e.printStackTrace();
                // Rollback on Error
                if (con != null) {
                    con.rollback();
                }
            } catch (SQLException ex) {
                ex.printStackTrace();
            }
        }
    }
}
```

snappify.com

📊 Metadata in JDBC

✅ Overview

In **JDBC (Java Database Connectivity)**, **metadata** refers to information about the structure of the database, its tables, columns, and other database objects. It provides a way for Java programs to gather information about the database without directly querying it.

JDBC provides two main types of metadata:

1. **Database Metadata** using the DatabaseMetaData interface.
2. **ResultSet Metadata** using the ResultSetMetaData interface.

1. Database Metadata (DatabaseMetaData)

📌 What is DatabaseMetaData?

- DatabaseMetaData provides information about the **database**, such as its version, driver details, tables, columns, and supported features.
- It is useful for dynamically exploring a database.

```
import java.sql.*;

public class DatabaseMetadataExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/mydatabase";
        String user = "root";
        String password = "1234";

        try (Connection con = DriverManager.getConnection(url, user, password)) {

            // Get Database Metadata
            DatabaseMetaData dbMetaData = con.getMetaData();

            System.out.println("Database Name: " + dbMetaData.getDatabaseProductName());
            System.out.println("Database Version: " + dbMetaData.getDatabaseProductVersion());
            System.out.println("Driver Name: " + dbMetaData.getDriverName());
            System.out.println("Driver Version: " + dbMetaData.getDriverVersion());
            System.out.println("Supports Transactions: " + dbMetaData.supportsTransactions());

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

🗂️ 2. ResultSet Metadata (ResultSetMetaData)

📌 What is ResultSetMetaData?

- ResultSetMetaData provides information about the **structure of a table** returned by a SQL query.
- It includes details about column names, data types, column sizes, and more.

RowSets in Java

✅ Overview

A **RowSet** is an extension of the ResultSet interface in **JDBC** that provides a more flexible and convenient way to handle tabular data. Unlike ResultSet, which requires an active database connection, **RowSets** can operate in both **connected** and **disconnected** modes.

- RowSets are part of the **javax.sql** package.
- They can be serialized and transferred between applications or tiers in a networked environment.
- Useful for **lightweight data manipulation** in Java applications.

Types of RowSets in Java

RowSet Type	Description	Use Case
JdbcRowSet	A connected RowSet that acts like a ResultSet.	When direct database access is required.
CachedRowSet	A disconnected RowSet that stores data in memory.	Ideal for offline data manipulation.

1. JdbcRowSet (Connected RowSet)

JdbcRowSet operates similarly to ResultSet but offers additional flexibility. It remains connected to the database while you scroll through data.

```
import javax.sql.rowset.JdbcRowSet;
import javax.sql.rowset.RowSetProvider;
import java.sql.SQLException;

public class JdbcRowSetExample {
    public static void main(String[] args) {
        try (JdbcRowSet rowSet = RowSetProvider.newFactory().createJdbcRowSet()) {

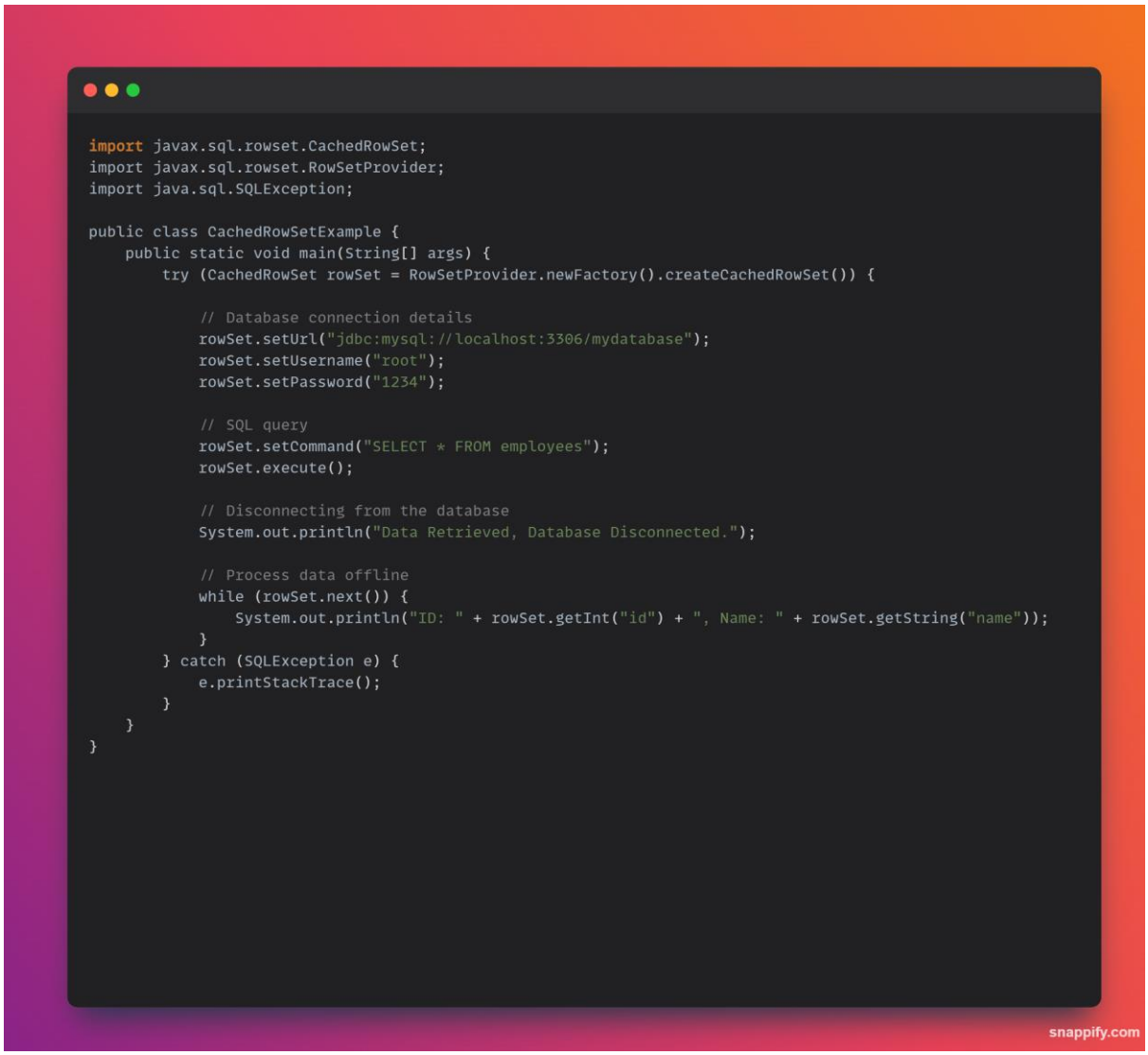
            // Database connection details
            rowSet.setUrl("jdbc:mysql://localhost:3306/mydatabase");
            rowSet.setUsername("root");
            rowSet.setPassword("1234");

            // SQL query
            rowSet.setCommand("SELECT * FROM employees");
            rowSet.execute();

            // Process data
            while (rowSet.next()) {
                System.out.println("ID: " + rowSet.getInt("id") + ", Name: " + rowSet.getString("name"));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

2. CachedRowSet (Disconnected RowSet)

CachedRowSet stores data in memory and operates without a persistent connection to the database. It is useful for offline data manipulation



Feature	RowSet	ResultSet
Connectivity	Can operate in both **connected** and **disconnected** modes.	Requires an active database connection to fetch and process data.
Data Manipulation	Supports offline data manipulation using **CachedRowSet.**	Data can only be manipulated while connected.
Scrollability	Supports bi-directional scrolling (move forward and backward).	Supports bi-directional scrolling only if `ResultSet.TYPE_SCROLL_SENSITIVE` or `ResultSet.TYPE_SCROLL_INSENSITIVE` is used.
Updatability	Can update data and sync back to the database.	Supports updates only for updatable ResultSets.
Serialization	Can be serialized and transferred over the network.	Cannot be serialized as it is connected to the database.
XML Support	Supports data read/write in **XML** using **WebRowSet.**	Does not support XML operations.
Ease of Use	Easier to use, with built-in functions for filtering and sorting using **FilteredRowSet.**	Requires manual implementation for filtering or sorting.
Connection Management	Efficient in managing connections as it disconnects after fetching data.	Requires persistent connection until data processing is complete.
Performance	Ideal for applications needing offline data manipulation.	Faster for large data sets when direct database interaction is required.
Use Case	Best for mobile applications, data exchange, or offline data processing.	Suitable for large-scale applications that process data in real-time.
Example Implementation	`RowSetProvider.newFactory().createCachedRowSet()`	`Statement.executeQuery()` returns a ResultSet.